

# **Cryptographic Hash Algorithm Litchi**

**Designer: Wenyi Jia<sup>1,2</sup>, Yupeng Liu<sup>1</sup>, Liang Bai<sup>1,2</sup>, Honggang Hu<sup>3</sup>, Changlun Li<sup>1,2</sup>, Yitong Liu<sup>1</sup>, Ye Peng<sup>1</sup>, Jiali Wu<sup>1</sup>**

**Completed by: <sup>1</sup>Xingtang Telecommunications Technology Co., Ltd. , <sup>2</sup>Beijing Key Laboratory of Post-Quantum Cryptography, <sup>3</sup>University of Science and Technology of China**

**Date: 2026-06-29**



## Contents

|                                              |    |
|----------------------------------------------|----|
| <b>1 Introduction</b>                        | 1  |
| <b>2 Algorithm Description</b>               | 1  |
| 2.1 Overview                                 | 1  |
| 2.1.1 Notation                               | 1  |
| 2.1.2 Word (Bit) Order and Conventions       | 2  |
| 2.2 Mode of Operation                        | 2  |
| 2.2.1 The Litchi Algorithm                   | 3  |
| 2.2.2 Padding                                | 4  |
| 2.3 Permutation P                            | 4  |
| 2.3.1 The State of Permutation P             | 4  |
| 2.3.2 Constant Addition $\iota$              | 5  |
| 2.3.3 Nonlinear Transformation $\chi$        | 5  |
| 2.3.4 Linear Transformation $\theta$         | 6  |
| 2.3.5 Transposition Transformation $\pi$     | 7  |
| 2.4 HASH Function                            | 7  |
| 2.5 XOF Function                             | 7  |
| <b>3 Design Rationale</b>                    | 8  |
| 3.1 The Design of the Mode of Operation      | 8  |
| 3.2 Design of Permutation P                  | 9  |
| 3.2.1 Design of Nonlinear Transformation     | 10 |
| 3.2.2 Design of Linear Transformation        | 12 |
| 3.2.3 Design of Transposition Transformation | 13 |
| 3.2.4 Design of Constant Addition            | 14 |
| 3.3 Parameter Selection Rationale            | 14 |
| <b>4 Security Claims and Analysis</b>        | 14 |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| 4.1 Theoretical Security .....                                         | 14        |
| 4.1.1 Security Analysis of the Mode of Operation.....                  | 14        |
| 4.1.2 Analysis of the Permutation P .....                              | 17        |
| 4.2 Concrete Security .....                                            | 27        |
| 4.3 Other Security Properties .....                                    | 27        |
| 4.3.1 Statistical Test Analysis .....                                  | 27        |
| 4.3.2 Length Extension Attacks.....                                    | 30        |
| <b>5 Performance Evaluation .....</b>                                  | <b>30</b> |
| 5.1 Performance Analysis.....                                          | 30        |
| 5.1.1 Design Efficiency of the Overall Structure .....                 | 30        |
| 5.1.2 Software and Hardware Efficiency of Core Components.....         | 31        |
| 5.2 Performance Testing .....                                          | 31        |
| 5.2.1 Software Performance Testing .....                               | 31        |
| 5.2.2 Hardware Performance Testing .....                               | 33        |
| <b>6 Features .....</b>                                                | <b>35</b> |
| 6.1 Innovativeness .....                                               | 35        |
| 6.2 Simplicity.....                                                    | 35        |
| 6.3 Flexibility.....                                                   | 36        |
| 6.4 Implementation Performance Advantages on Different Platforms ..... | 36        |
| <b>7 Comparison with SHA-3 Algorithm .....</b>                         | <b>37</b> |
| <b>8 References.....</b>                                               | <b>37</b> |

# 1 Introduction

In response to the call for cryptographic hash algorithms submissions under the Next-generation Commercial Cryptographic Algorithm Program (NGCC) held by the Institute of Commercial Cryptography Standards, we have developed the cryptographic hash algorithm Litchi. Litchi comprises three hash algorithms with output lengths of 512, 768, and 1024 bits, respectively, and one extendable-output function (XOF). Based on the different hash output lengths, the three hash algorithms are named Litchi-512, Litchi-768, and Litchi-1024, respectively, and the extendable-output function is named Litchi-XOF. The specific characteristics of the four algorithms are given in Table 1-1.

Table 1-1 Specific Characteristics of Litchi-512/768/1024/XOF

| Algorithm Name | Input Message Bit Length | Output Hash Bit Length |
|----------------|--------------------------|------------------------|
| Litchi-512     | $\leq 2^{64}-1$          | 512                    |
| Litchi-768     | $\leq 2^{64}-1$          | 768                    |
| Litchi-1024    | $\leq 2^{64}-1$          | 1024                   |
| Litchi-XOF     | $\leq 2^{64}-1$          | $\leq 2^{64}-1$        |

The Litchi hash algorithm is capable of resisting known classical computational attacks and quantum computational attacks. It can be implemented efficiently on a wide range of software and hardware platforms, featuring security, efficiency, and flexibility.

This document provides a description of the algorithm, elaborates its design rationale, presents a preliminary security analysis, and reports performance evaluations.

## 2 Algorithm Description

### 2.1 Overview

The Litchi algorithm is designed based on the Sponge- $F^{pd}$  structure proposed in [GHJ+25], with the internal compression function employing a permutation of state size 1600 bits. Based on the Litchi algorithm, we present the Litchi-512, Litchi-768, and Litchi-1024 hash algorithms, with output lengths of 512, 768, and 1024 bits, respectively, and simultaneously design the extendable-output function Litchi-XOF.

#### 2.1.1 Notation

|          |                                                |
|----------|------------------------------------------------|
| 0x       | Prefix indicating a hexadecimal number.        |
| =        | Assignment operator, may also denote equality. |
| +        | Arithmetic addition operator.                  |
| $\oplus$ | Bitwise XOR operator.                          |

|                   |                                                                                   |
|-------------------|-----------------------------------------------------------------------------------|
| $\&$              | Bitwise AND operator.                                                             |
| $\text{mod}$      | Integer modulo operator.                                                          |
| $\ $              | String concatenation operator.                                                    |
| $\#$              | Number of elements in a set.                                                      |
| $\min$            | Returns the minimum value among a set of elements.                                |
| $\lll n$          | Operator for cyclic left shift of operand by $n$ bits.                            |
| $\ggg n$          | Operator for cyclic right shift of operand by $n$ bits.                           |
| $\ll n$           | Operator for left shift of operand by $n$ bits, filling lower bits with 0.        |
| $\gg n$           | Operator for right shift of operand by $n$ bits, filling higher bits with 0.      |
| $\{0, 1\}^n$      | Space of $n$ -bit strings.                                                        |
| $A^T$             | Transpose operation of matrix (or vector) $A$ .                                   |
| $\text{MSB}_n(x)$ | The $n$ most significant bits of $x$ .                                            |
| $\text{LSB}_n(x)$ | The $n$ least significant bits of $x$ .                                           |
| $0^n$             | All-zero string of $n$ bits.                                                      |
| $[i]_n$           | The $n$ -bit binary representation string of the non-negative integer $i < 2^n$ . |
| $\lceil \rceil$   | Ceiling operator.                                                                 |
| $N$               | Number of iterations of permutation $P$ , set to 16.                              |
| $\text{RC}_i$     | 64-bit round constant for round $i$ .                                             |

### 2.1.2 Word (Bit) Order and Conventions

Data variables are represented with the most significant bit (or block) on the left and the least significant bit (or block) on the right, numbered from left to right starting from 0. For example, for an  $n$ -bit number, bits from highest to lowest (left to right) are referred to as bit 0, bit 1, bit 2, ..., bit  $n-1$ . When a variable is divided into several blocks, the leftmost (most significant) block consists of the most significant part of the original data, and the rightmost (least significant) block consists of the least significant part of the original data.

Data adopts big-endian byte order. When the input (or output) is a word or byte stream, the lower address corresponds to higher-order data (i.e., left side), and the higher address corresponds to lower-order data (i.e., right side). Every 8 (or 4) bytes form a 64-bit (or 32-bit) word.

Unless otherwise specified, all polynomials, vectors, and matrices in the following text are defined over the binary field.

## 2.2 Mode of Operation

The mode of operation of the Litchi algorithm consists of three phases: **padding**, **absorbing**, and **squeezing**.

**Padding Phase:** Based on the security level parameter  $c$ , the input message  $M$  is

padding to meet the specified length requirements, resulting in the padded message  $M_{\text{pad}}$  which is split into  $l$  equal-length blocks  $M_0||M_1||\dots||M_{l-1}$ .

**Absorbing Phase:** Starting from the initial state  $S_0$ , the padded message blocks  $M_i$  are iteratively compressed to update the hash state  $S_i$ .

**Squeezing Phase:** Using the hash state produced in the absorbing phase as input, an extended output is performed according to the output length  $|h|$  to obtain the hash value  $h = \text{MSB}_{|h|}(h_1||h_2||\dots||h_{\lceil |h|/c \rceil})$ .

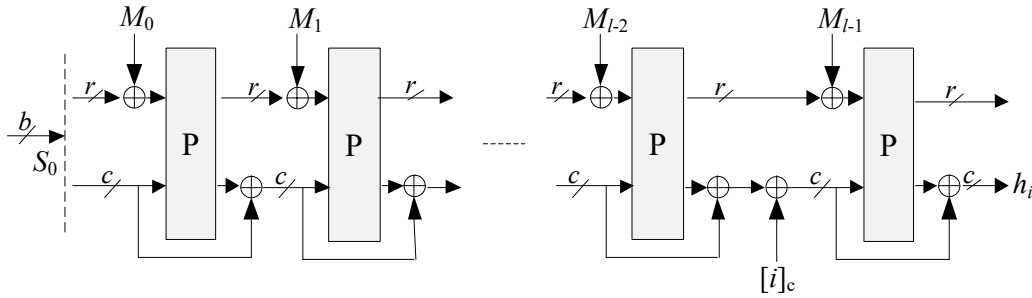


Figure 2-1 Litchi mode of operation

The Litchi mode of operation is shown in Figure 2-1, where  $P$  is a permutation from  $\{0,1\}^b$  to  $\{0,1\}^b$ , with  $b=r+c$ . Here,  $M_i (0 \leq i \leq l-1)$  has length  $r$  bits, and  $h_i (1 \leq i \leq \lceil |h|/c \rceil)$  has length  $c$  bits. Unlike the  $\text{Sponge-F}^{\text{P,pd}}$  structure, when generating variable-length hash outputs, the Litchi mode of operation employs a counter based parallel method instead of a serial iterative approach.

### 2.2.1 The Litchi Algorithm

For the Litchi algorithm,  $b=1600$ . To support different security levels and functional outputs, we use the security parameters  $c$  and the function identifier  $fid$  to control Litchi.  $fid=0$  is used for hash algorithms, and  $fid=1$  is used for the extendable output function (XOF).

The input message  $M$  has bit length  $|M|$ , satisfying  $|M| \leq 2^{64}-1$ . The output hash value  $h$  has bit length  $|h|$ . When  $fid=0$ , it is required that  $|h| \leq c-64$ , when  $fid=1$ , it is required that  $|h| \leq 2^{64}-1$ .

$h = \text{Litchi}[c, fid](M, |h|)$  is computed as follows:

(1) Compute  $M_0||M_1||\dots||M_{l-1} = \text{Pad}[c](M)$ ;

(2)  $S_0 = [2^{63}fid + c]_{1600}$ ;

(3) For  $1 \leq i \leq l-1$ , execute:

$$S_i = P(S_{i-1} \oplus (M_{i-1}||0^c)) \oplus (0^r||\text{LSB}_c(S_{i-1}));$$

(4) For  $1 \leq i \leq \lceil |h|/c \rceil$ , execute:

- 1)  $ST = S_{l-1} \oplus (M_{l-1} \parallel [i]_c)$ ;
- 2)  $h_i = \text{LSB}_c(P(ST)) \oplus \text{LSB}_c(ST)$ ;
- (5) Return  $h = \text{MSB}_{|h|}(h_1 \parallel h_2 \parallel \dots \parallel h_{\lceil |h|/c \rceil})$ .

### 2.2.2 Padding

Pad is the message padding function controlled by the security parameter  $c$ . The padding appends a single 1 bit to the message  $M$ , followed by the minimum number of 0 bits such that the length of the padded message  $M_{\text{pad}}$  is a multiple of  $1600-c$  bits.  $M_{\text{pad}} = \text{Pad}[c](M)$  is computed as follows:

$$M_{\text{pad}} = M_0 \parallel M_1 \parallel \dots \parallel M_{l-1} = M \parallel 1 \parallel 0^{r-1 - ((|M|) \bmod r)}.$$

where  $|M|$  is the bit length of message  $M$ ,  $r = 1600 - c$ ,  $l = \lceil (|M| + 1) / r \rceil$ , and each  $M_i$  ( $0 \leq i \leq l-1$ ) has length  $r$  bits.

## 2.3 Permutation P

The Litchi hash algorithm uses a permutation  $P$  from  $\{0,1\}^{1600}$  to  $\{0,1\}^{1600}$ . Permutation  $P$  adopts an SPN (Substitution-Permutation Network) structure and is formed by iterating the round function  $f$  for  $N$  rounds. The input of permutation  $P$  is a 1600 bit state  $A$ , and the output is a 1600 bit state  $B$ , with  $B = P(A)$  executed as follows:

- (1)  $A_0 = A$ ;
- (2) For  $i = 0$  to  $N-1$ , execute:  
 $A_{i+1} = f(A_i, i_r)$ ;
- (3) Return  $B = A_N$ .

The round function  $f$  takes as input a 1600 bit state  $A_i$  and a round index  $i_r$ .  $f$  consists of four components: the constant addition  $\mathfrak{u}$ , the nonlinear transformation  $\chi$ , the linear transformation  $\theta$ , and the transposition transformation  $\pi$ . Specifically:  $f(A_i, i_r) = \pi(\theta(\chi(\mathfrak{u}(A_i, i_r))))$ .

### 2.3.1 The State of Permutation P

The 1600-bit state of permutation  $P$  is organized as 5 planes of 320 bits each, and each plane consists of 5 lanes of 64 bits each. Thus, the state of  $P$  can be viewed as a  $5 \times 5 \times 64$  cube.

Let  $A$  denotes the entire 1600-bit state. For  $0 \leq i \leq 4$ , let  $A[i]$  denotes the  $i$ -th plane of  $A$ , which is 320 bits. For  $0 \leq j \leq 4$ , let  $A[i][j]$  denotes the  $j$ -th lane of  $A[i]$ , which is 64 bits. For  $0 \leq k \leq 63$ , let  $A[i][j][k]$  denotes the  $k$ -th bit of  $A[i][j]$ , which is 1 bit.

Thus,



$$\begin{aligned}
A &= A[0] \parallel A[1] \parallel \dots \parallel A[4]; \\
A[i] &= A[i][0] \parallel A[i][1] \parallel \dots \parallel A[i][4], \quad 0 \leq i \leq 4; \\
A[i][j] &= A[i][j][0] \parallel A[i][j][1] \parallel \dots \parallel A[i][j][63], \quad 0 \leq i \leq 4, \quad 0 \leq j \leq 4.
\end{aligned}$$

### 2.3.2 Constant Addition $\iota$

The constant addition  $\iota$  adds the 64-bit round constant  $RC_r$  of round  $r$  to the lane  $A[0][0]$ , while keeping all other lanes unchanged.  $B = \iota(A, r)$  is executed as follows:

- (1)  $B = A$ ;
- (2)  $B[0][0] = A[0][0] \oplus RC_r$ ;
- (3) Return  $B$ .

The round constants  $RC_r$  ( $0 \leq r \leq N-1$ ) are computed from constant  $C$ , with specific values listed in Table 2-1. Constant  $C$  is the first 64 bits of the fractional part of the natural constant  $e$ , represented in hexadecimal as:

$$C = 0xb7e151628aed2a6a.$$

$RC_r = 2^r \otimes C$ , where “ $\otimes$ ” denotes multiplication in the finite field  $GF(2^{64}) = GF(2)[x]/(x^{64} + x^{63} + x^{62} + x^{54} + x^{40} + x^{35} + x^{15} + x + 1)$ .

Table 2-1 Round Constants  $RC_r$

| Round $r$ | Round Constant $RC_r$ | Round $r$ | Round Constant $RC_r$ |
|-----------|-----------------------|-----------|-----------------------|
| 0         | 0xb7e151628aed2a6a    | 8         | 0x5dd1971aed536b16    |
| 1         | 0xaf82a3cd15dad4d7    | 9         | 0xbba32e35daa6d62c    |
| 2         | 0x9f4546922bb529ad    | 10        | 0xb7065d63b54d2c5b    |
| 3         | 0xfeca8c2c576ad359    | 11        | 0xae4cbbcf6a9ad8b5    |
| 4         | 0x3dd51950aed526b1    | 12        | 0x9cd97696d5353169    |
| 5         | 0x7baa32a15daa4d62    | 13        | 0xf9f2ec25aa6ae2d1    |
| 6         | 0xf7546542bb549ac4    | 14        | 0x33a5d94354d545a1    |
| 7         | 0x2ee8cb8d76a9b58b    | 15        | 0x674bb286a9aa8b42    |

### 2.3.3 Nonlinear Transformation $\chi$

The nonlinear transformation  $\chi$  updates the state  $A$  by applying 320 identical 5-bit S-boxes in parallel, producing state  $B$ . The lookup table of the S-box is given in Table 2-2, and its circuit implementation is shown in Figure 2-2.

Table 2-2: S-box Lookup Table

| x | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
|---|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|

|      |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| S(x) | 0  | 2  | 4  | 23 | 8  | 10 | 14 | 13 | 16 | 18 | 20 | 15 | 28 | 30 | 26 | 17 |
| x    | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| S(x) | 1  | 19 | 5  | 6  | 9  | 27 | 31 | 12 | 25 | 11 | 29 | 22 | 21 | 7  | 3  | 24 |

The nonlinear transformation  $B=\chi(A)$  is equivalently expressed as:

- (1)  $B[0] = A[0]\&A[2]\&A[3] \oplus A[2]\&A[3]\&A[4] \oplus A[0]\&A[4] \oplus A[3]\&A[4] \oplus A[1];$
- (2)  $B[1] = A[1]\&A[3]\&A[4] \oplus A[0]\&A[1] \oplus A[2];$
- (3)  $B[2] = A[1]\&A[2] \oplus A[3];$
- (4)  $B[3] = A[2]\&A[3] \oplus A[4];$
- (5)  $B[4] = A[3]\&A[4] \oplus A[0];$
- (6) Return B.

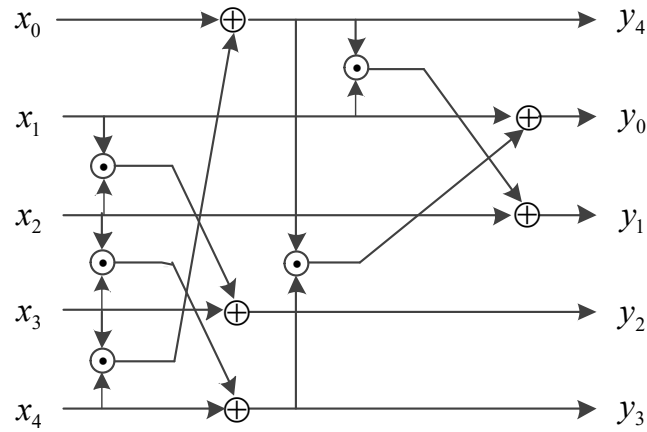


Figure 2-2: Circuit diagram of the S-box

### 2.3.4 Linear Transformation $\theta$

The linear transformation  $\theta$  updates each plane  $A[i]$  of state A separately through the linear transformation  $\theta_i$ , for  $0 \leq i \leq 4$ .  $B=\theta(A)$  is executed as follows:

- (1) For  $0 \leq i \leq 4$ , execute:  
 $B[i] = \theta_i(A[i]);$
- (2) Return B.

$\theta_i(0 \leq i \leq 4)$  is a 320-bit linear permutation determined by the parameterized linear permutation Theta. Specifically:

- $$\begin{aligned} \theta_0 &= \text{Theta} [31, 1, 4, 21, 22]; \\ \theta_1 &= \text{Theta} [19, 12, 15, 34, 13]; \\ \theta_2 &= \text{Theta} [28, 35, 58, 9, 23]; \\ \theta_3 &= \text{Theta} [55, 18, 5, 27, 44]; \\ \theta_4 &= \text{Theta} [14, 43, 53, 20, 25]. \end{aligned}$$

Theta[ $t_0, t_1, t_2, t_3, t_4$ ] is a 320-bit linear permutation determined by 5 parameters ( $t_0, t_1, t_2, t_3, t_4$ ),  $Y = \text{Theta}[t_0, t_1, t_2, t_3, t_4](X)$  is executed as follows:

- 1) Divide  $X$  into 5 blocks of 64 bits  $X_i$  ( $0 \leq i \leq 4$ ), i.e.,  $X = X_0 || X_1 || X_2 || X_3 || X_4$ ;
- 2)  $E = X_0 \oplus X_1 \oplus X_2 \oplus X_3 \oplus X_4$ ;
- 3)  $F = (X_0 \lll t_0) \oplus (X_1 \lll t_1) \oplus (X_2 \lll t_2) \oplus (X_3 \lll t_3) \oplus (X_4 \lll t_4)$ ;
- 4) For  $0 \leq i \leq 4$ , execute:  

$$Y_i = X_i \oplus (E \ggg t_i) \oplus F$$
;
- 5)  $Y = Y_0 || Y_1 || Y_2 || Y_3 || Y_4$ .

### 2.3.5 Transposition Transformation $\pi$

The transposition transformation  $\pi$  performs a permutation of the 25 lanes of state  $A$ .  $B = \pi(A)$  is executed as follows:

- (1) For  $0 \leq i \leq 4$  and  $0 \leq j \leq 4$ , execute:  

$$B[i][j] = A[(i+2j) \bmod 5][(3i+3j) \bmod 5]$$
;
- (2) Return  $B$ .

## 2.4 HASH Function

The hash functions Litchi-512, Litchi-768, and Litchi-1024 are constructed based on the Litchi algorithm. Their input is a message  $M$  of length less than  $2^{64}$  bits, computed as follows:

$$\begin{aligned} \text{Litchi-512}(M) &= \text{Litchi}[576, 0](M, 512); \\ \text{Litchi-768}(M) &= \text{Litchi}[832, 0](M, 768); \\ \text{Litchi-1024}(M) &= \text{Litchi}[1088, 0](M, 1024). \end{aligned}$$

## 2.5 XOF Function

The extendable-output function Litchi-XOF is constructed based on the Litchi algorithm. Its input is a message  $M$  of length less than  $2^{64}$  bits, and its output length  $|xof|$  is less than  $2^{64}$  bits, computed as follows:

$$\text{Litchi-XOF}(M, |xof|) = \text{Litchi}[1024, 1](M, |xof|).$$

*Note: Since the call for submissions did not specify concrete requirements for extendable-output functions, we provide an example of an XOF here, primarily considering the efficiency of hash output and security strength, while also accommodating common message input lengths such as 256 and 512 bits. In practice, the parameter  $c$  can be adjusted according to security strength requirements and efficiency needs to give other versions of the extendable-output function.*

### 3 Design Rationale

#### 3.1 The Design of the Mode of Operation

The widely used Sponge construction [BDPV07] can only achieve  $c/2$ -bit security against (second) preimage attacks [BDPV08]. When the hash output length is 1024 bits, the permutation  $P$  used in the Sponge construction must exceed 2048 bits. Moreover, for efficiency reasons, the rate  $r$  cannot be too small. This necessitates the design of a very large permutation  $P$ , which poses significant difficulties for both design and analysis, while also severely impacting implementation efficiency due to the large capacity  $c$ .

The Sponge- $F^{P,pd}$  structure proposed in [GHJ+25] enhances resistance against preimage and second preimage attacks by adding feed forward to the  $c$ -bit capacity part of the Sponge construction. When the hash output length is  $h$  (with  $h \leq c$ ), the Sponge- $F^{P,pd}$  structure achieves  $h/2$ -bit collision resistance,  $h$ -bit preimage resistance, and second preimage resistance of up to  $\min(h, c - \log_2 l)$  bits, where  $l$  is the number of blocks in the second-preimage challenge message. Moreover, the Sponge- $F^{P,pd}$  structure exhibits strong security against quantum attacks. Due to its strong (second) preimage resistance, the Sponge- $F^{P,pd}$  structure can reduce the required size of the permutation  $P$ . For example, with a 1600-bit permutation  $P$  and  $c=1088$ , the preimage and second preimage security strength reaches 1024 bits, satisfying the design requirement for a 1024-bit hash output.

To address the efficiency impact caused by the serial iteration of Sponge- $F^{P,pd}$ , and taking into account the non-zero constant XORed into the final message block to resist length extension attacks, we modify the Sponge- $F^{P,pd}$  structure and design the Litchi mode of operation. The main modifications are as follows:

(1) Parallel output generation: A counter is used as a constant to generate different hash output blocks  $h_i$  in parallel, improving the efficiency of hash output.

(2) Domain separation: A function identifier  $fid$  and a security parameter  $c$  are introduced into the initial vector  $S_0$  to distinguish between different functions and security levels, ensuring that the same message yields different hash outputs when used for different purposes or at different security levels.

In the concrete algorithm design, we use a single 1600-bit permutation  $P$ . By varying the security capacity  $c$ , we design three hash algorithms with security levels of 512, 768, and 1024 bits: Litchi-512, Litchi-768, and Litchi-1024. Simultaneously, we design the Litchi-XOF algorithm with security parameter  $c=1024$ , achieving a security strength of 512 bits.

In summary, the Litchi mode of operation designed according to the above principles achieves higher efficiency than the Sponge construction when attaining the same level of security. For example, for a hash output length of 512 bits, the message processing efficiency is nearly doubled. Moreover, when used as an extendable-output function, the parallel output processing further improves efficiency.

### 3.2 Design of Permutation P

The permutation P of the Litchi hash algorithm is designed with a focus on bitsliced implementation and adopts an SPN structure. The round function consists of the constant addition  $\iota$ , the nonlinear transformation  $\chi$ , the linear transformation  $\theta$ , and the transposition transformation  $\pi$ . Each component is designed to be as simple as possible, and through the interplay of these components, a high level of security is achieved with a relatively low number of rounds.

The linear transformation  $\theta$  is the most innovative aspect and design core of permutation P. It is designed based on a parameterized 320-bit linear transformation  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$ .  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$  has the same implementation depth as Keccak's [BDPV11, DM15] linear transformation, but its branch number is increased from 4 to 12. By selecting different parameters, different linear transformations can be designed. The design of permutation P revolves around the linear transformation  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$ , with careful design of other cryptographic components and related parameters.

The design of permutation P draws inspiration from the Ascon algorithm [DEMS21]. Ascon's permutation P divides a 320-bit state into 5 lanes of 64 bits each, using simple linear transformations controlled by different parameters for different lanes to achieve diffusion. The confusion between lanes is provided by a nonlinear transformation (a 5-bit S-box) whose differential and linear branch numbers are both 3, enabling the propagation of differences and linear masks across more lanes.

The permutation P of Litchi divides the 1600-bit state into 5 planes of 320 bits each, using simple linear transformations controlled by different parameters for different planes to achieve diffusion. The confusion between planes is provided by a nonlinear transformation (a 5-bit S-box). Unlike the design of the S-box in Ascon's permutation P, aiming for greater simplicity and efficiency, we did not adopt an S-box with branch number 3; instead, we chose an S-box with branch number 2 and with as few trails as possible. At the same time, we added a transposition transformation  $\pi$  to exchange the 64-bit lanes between different planes, achieving faster inter-plane diffusion. Moreover, the transposition transformation  $\pi$  incurs no additional cost in either software or hardware implementation.

Through careful parameter selection, the permutation P of Litchi achieves full diffusion in 3 rounds and possesses strong resistance against differential and linear attacks.

Below, we describe in detail the design principles and cryptographic properties of each component of the round function.

### 3.2.1 Design of Nonlinear Transformation

The nonlinear transformation  $\chi$  mainly provides confusion across the corresponding bits of the five planes of state A. Based on a trade-off between security and efficiency considerations, the design of the nonlinear transformation took into account the algebraic degree of the S-box, the maximum differential probability, the maximum linear bias, and the diffusion properties of differentials and linear masks. At the same time, to enable efficient bitsliced implementation, the S-box must have a compact algebraic expression. Specifically, the properties of the S-box used in the nonlinear transformation  $\chi$  are as follows:

- (1) The algebraic degree is 3; the inverse S-box also has algebraic degree 3;
- (2) Maximum differential probability is  $2^{-2}$ ;
- (3) Maximum linear bias is  $2^{-2}$ ;
- (4) The differential branch number is 2. In the difference distribution table (see Table 3-1), the number of entries where both input and output have Hamming weight 1 is 5;
- (5) The linear branch number is 2. In the linear approximation table (see Table 3-2), the number of entries where both input and output have Hamming weight 1 is 5;
- (6) The S-box can be implemented with only 5 AND gates and 5 XOR gates; see Section 5.2.1 “Software Performance Testing” for details.

Table 3-1 Difference Distribution Table :  $DDT[\alpha, \beta] = \#\{x | S(x \oplus \alpha) \oplus S(x) = \beta\}$

|    | 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |   |
|----|----|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|
| 0  | 32 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |   |
| 1  | 0  | 0 | 8 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 0  |   |
| 2  | 0  | 0 | 0 | 0 | 8 | 0 | 4 | 2 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 2  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 2  |   |
| 3  | 0  | 0 | 0 | 0 | 8 | 4 | 4 | 2 | 0 | 0 | 0  | 0  | 0  | 4  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 2  |   |
| 4  | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8 | 0 | 4  | 0  | 8  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  |   |
| 5  | 0  | 0 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 4  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 4  | 0  |   |
| 6  | 0  | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2  | 0  | 4  | 0  | 2  | 2  | 0  | 4  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 4  | 2  | 2 |
| 7  | 0  | 4 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2  | 0  | 4  | 4  | 2  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  | 2 |
| 8  | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  |   |
| 9  | 0  | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  |   |
| 10 | 0  | 0 | 0 | 2 | 0 | 4 | 0 | 0 | 0 | 0 | 4  | 2  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 2  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 4  | 0  | 0  | 0  |   |
| 11 | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 0  | 0  | 0  | 0  | 4  | 2  | 4  | 4  | 0  | 0  | 0  | 0  | 4  | 2  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 2  |   |

|    |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 12 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 4 | 0 | 4 | 0 | 4 | 0 |   |   |   |
| 13 | 0 | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 4 | 0 |   |   |
| 14 | 0 | 2 | 2 | 0 | 0 | 2 | 2 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 0 | 2 | 0 | 0 | 2 | 2 | 0 | 0 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 2 |   |   |
| 15 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 2 | 2 | 0 | 2 | 2 | 2 | 0 | 0 | 2 | 2 | 2 | 0 | 2 | 2 | 2 | 2 |   |   |
| 16 | 0 | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8 | 0 | 0 | 0 | 0 | 0 |   |   |
| 17 | 0 | 0 | 8 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 0 | 0 | 8 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 0 |   |   |
| 18 | 0 | 0 | 0 | 0 | 8 | 4 | 4 | 2 | 0 | 0 | 0 | 0 | 0 | 4 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 |   |   |
| 19 | 0 | 0 | 0 | 0 | 8 | 0 | 4 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 4 | 4 | 2 | 0 | 0 | 0 | 0 | 0 | 4 | 0 | 2 |   |   |
| 20 | 0 | 0 | 0 | 0 | 0 | 4 | 0 | 4 | 0 | 4 | 0 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 0 | 4 | 0 | 4 | 0 | 4 | 0 | 0 | 0 | 0 |   |   |
| 21 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 8 | 0 | 0 | 4 | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 0 | 0 |   |   |
| 22 | 0 | 4 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2 | 0 | 4 | 4 | 2 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 2 |   |
| 23 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2 | 0 | 4 | 0 | 2 | 2 | 0 | 4 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 4 | 2 | 2 |   |
| 24 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 |   |
| 25 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 4 | 2 | 0 | 0 | 4 | 2 | 0 | 0 | 4 | 2 | 0 | 0 | 4 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 2 |   |
| 26 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 2 | 0 | 0 | 0 | 0 | 0 | 4 | 2 | 4 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 4 | 4 | 0 | 0 | 0 |   |
| 27 | 0 | 4 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 4 | 0 | 0 | 0 | 0 | 4 | 2 | 4 | 0 | 0 | 0 | 0 | 0 | 4 | 2 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 2 |   |
| 28 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 2 |   |
| 29 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 2 |   |
| 30 | 0 | 0 | 2 | 2 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 2 | 2 | 0 | 0 | 2 | 2 | 0 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 0 |   |
| 31 | 0 | 2 | 2 | 2 | 0 | 2 | 2 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 2 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 2 | 2 | 0 | 2 | 2 | 2 | 2 | 2 | 0 | 2 | 0 |

Table 3-2 Linear Approximation Table :  $LAT[\alpha, \beta] = \#\{x | \alpha^T \cdot x = \beta^T \cdot S(x)\} - 16$

|    | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0  | 16 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 1  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | -4 | 4  | 0  | 0  | 0  | 0  | 4  | -4 | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | -4 | -4 |
| 2  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | -4 | -4 | 4  | -4 |
| 3  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | -4 | -4 | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 4  | 0  | 0  | 4  | -4 | 0  | 0  |
| 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | -8 | 0  | 0  | 0  |
| 5  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 4  | -4 | 0  | 0  | 0  | 0  | -4 | 4  | -4 | -4 | 4  | -4 | 0  | 0  | 0  | 0  | 4  | 4  |
| 6  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | -4 | -4 | 4  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 |
| 7  | 0  | 0  | -8 | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 4  | -4 | 0  | 0  |
| 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 8  | 8  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 9  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | -4 | -4 | 4  | -4 | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | -4 | 4  | 0  | 0  |
| 10 | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | 4  | 4  | -4 | 4  |
| 11 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 4  | 0  | 0  | 4  | -4 | 4  | -4 | 4  | 4  | 4  | -4 | 4  | 4  | 0  | 0  | -4 | -4 | 0  | 0  | 4  | 4  |
| 12 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 8  | 0  | 0  | 0  | 0  | 0  | 0  |
| 13 | 0  | 0  | 0  | 0  | 0  | 0  | -8 | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 4  | 4  | 4  | -4 | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  |
| 14 | 0  | 0  | 0  | 0  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 |
| 15 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 4  | -4 | -4 | 4  | -4 | -4 | 4  | -4 | 4  | 4  | 0  | 0  | 4  | 4  | 0  | 0  | 4  | 4  |
| 16 | 0  | 8  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 0  | 4  | 4  | 4  | 0  | 0  | -4 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | -4 | 4  | 0  | 0  | 0  | -4 | -4 | 0  |
| 17 | 0  | 8  | 0  | 8  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | -4 |
| 18 | 0  | 8  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | -4 | -4 | 4  | 0  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 4  | -4 | 0  | 0  | -4 | 0  | 0  | 4  |
| 19 | 0  | -8 | 0  | 8  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 0  |
| 20 | 0  | 0  | 0  | 8  | 0  | 4  | 0  | -4 | 4  | 4  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 0  | 0  | -4 | 4  | 4  | 0  | 0  | 4  |
| 21 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 4  | 4  | 4  | 4  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | -4 | 4  | -4 | 4  | -4 | 0  | 4  | 0  |
| 22 | 0  | 0  | 0  | -8 | 0  | 4  | 0  | 4  | 4  | 4  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | -4 | 4  | 0  | 4  | -4 | 0  |
| 23 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | -4 | -4 | 4  | 4  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | -4 | 4  | 4  | -4 | 0  | 4  | 0  | 4  | 0  |
| 24 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | -4 | 4  | -4 | 0  | 0  | -4 | 4  | 4  | -4 | 4  | 4  | 0  | -4 | 0  | 0  | 0  | 4  | 4  | 4  | 0  | 0  | -4 |
| 25 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 0  | 8  | 0  | 8  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  |
| 26 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 0  | 4  | -4 | -4 | 0  | 0  | 4  | 4  | 4  | -4 | 4  | 0  | 4  | 0  | 4  | 0  | 0  | -4 | -4 | 0  | 4  | 4  | 0  |
| 27 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 0  | -8 | 0  | 8  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  |
| 28 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | -4 | 4  | 0  | 0  | 0  | 4  | 4  | 0  | -4 | 4  | 4  | 4  | 4  | 0  | -4 | 0  | 4  | 4  | 0  | 0  | 0  | 4  | -4 | 0  |
| 29 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | -4 | 4  | -4 | 4  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 4  | 4  | 4  | 4  | -4 | 0  | 4  | 0  |
| 30 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | -4 | 4  | 0  | 0  | 0  | 4  | -4 | 0  | 4  | -4 | -4 | -4 | 0  | 4  | 0  | 4  | 4  | 4  | 0  | 0  | 4  | 0  | 0  | 4  |
| 31 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 4  | -4 | -4 | 4  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | -4 | -4 | 4  | 4  | 0  | 4  | 0  | 4  | 0  |

To enhance resistance against algebraic attacks, the algebraic degree of the S-box is required to be no less than 3. Since the linear transformation  $\theta$  of

permutation  $P$  operates on each plane of state  $A$ , while the nonlinear transformation  $\chi$  mainly provides confusion across the corresponding bits of the five planes, if the S-box has good diffusion properties, it can enhance the resistance of permutation  $P$  against differential and linear attacks. However, if both the differential and linear branch numbers of the S-box reached 3, the complexity of the S-box expression would increase significantly, incurring a high implementation cost. As a compromise, although the designed S-box does not achieve branch number 3, it has as few points as possible with branch number 2.

### 3.2.2 Design of Linear Transformation

The linear transformation  $\theta$  updates each plane  $A[i]$  of state  $A$  separately through the linear transformation  $\theta_i$ , for  $0 \leq i \leq 4$ . Each linear transformation  $\theta_i$  is a 320-bit linear permutation determined by the parameterized linear permutation  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$ , where  $t_i$  are integers from 0 to 63.

$V = \text{Theta}[t_0, t_1, t_2, t_3, t_4](U)$  is a 320-bit linear permutation controlled by parameters  $(t_0, t_1, t_2, t_3, t_4)$ . The 320-bit input is divided into 5 blocks of 64 bits, i.e.,  $U = U_0 || U_1 || U_2 || U_3 || U_4$ ,  $V = V_0 || V_1 || V_2 || V_3 || V_4$ . Using the polynomial representation over the ring  $\text{GF}(2)[x]/(x^{64}+1)$ , it can be expressed as:

$$V_i = U_i + \sum_{j=0}^4 (x^{-t_i} + x^{t_j})U_j, 0 \leq i \leq 4$$

Theta satisfies the following properties:

(1) Symmetry:  $\text{Theta}^T = \text{Theta}$ ;

(2) Invertibility: its inverse transformation is

$$U_i = V_i + (\sum_{k=0}^4 x^{-t_k} + \sum_{k=0}^4 x^{t_k} + \sum_{k=0}^4 x^{-t_k} \sum_{k=0}^4 x^{t_k})^{-1} \sum_{j=0}^4 (1 + (1 + \sum_{k=0}^4 x^{t_k})x^{-t_i} + (1 + \sum_{k=0}^4 x^{-t_k})x^{t_j} + x^{-t_i+t_j})V_j$$

(3) When specific parameters are selected, both differential branch number and linear branch number can reach 12. Each  $\theta_i$  achieves a differential branch number and a linear branch number of 12.

To achieve branch number 12, the parameter selection in  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$  must satisfy the following requirements:

- ①  $2t_i \neq 2t_j \pmod{64}$ , for arbitrary distinct  $i, j$ , and  $0 \leq i, j \leq 4$ ;
- ②  $t_i \neq -t_j \pmod{64}$ , for arbitrary  $i, j$ , and  $0 \leq i, j \leq 4$ ;
- ③  $t_i + t_j \neq 2t_k \pmod{64}$ , for arbitrary distinct  $i, j, k$ , and  $0 \leq i, j, k \leq 4$ ;



- ④  $t_i + t_j \neq t_k + t_l \pmod{64}$ , for arbitrary distinct  $i, j, k, l$ , and  $0 \leq i, j, k, l \leq 4$ .

The hardware implementation depth of Theta is the same as that of Keccak's linear transformation. The software implementation requires only 18 64-bit XOR operations and 10 cyclic shifts, while the branch number is increased from 4 to 12, significantly enhancing the resistance of permutation P against differential, linear, and other attacks. The STCPM scheme proposed in [HWL24] can also achieve a differential and linear branch number of 12, but its computational depth and complexity are higher than those of the Theta transformation.

### 3.2.3 Design of Transposition Transformation

The primary function of the transposition transformation  $\pi$  is to perform a permutation of the lanes, ensuring that the five lanes on the same plane of state A, after the transposition transformation  $\pi$ , reside in five different planes, thereby achieving rapid diffusion. At the same time, to preserve the branch number of the linear transformation  $\theta_i$ , any two lanes from the same plane of state A, after the transposition transformation  $\pi$ , must not be located in the same column, i.e., no two lanes shall have corresponding bits processed by the same S-box.

The lane position transformation in transposition transformation  $\pi$  can be viewed as the following matrix operation over GF(5). The index  $(i, j)$  of a lane, after  $\pi$  transformation, becomes  $(i', j')$ . The computation is:

$$\begin{pmatrix} i' \\ j' \end{pmatrix} = M \begin{pmatrix} i \\ j \end{pmatrix} = \begin{pmatrix} 4 & 4 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} i \\ j \end{pmatrix}$$

Transposition transformation  $\pi$  satisfies the following properties:

- (1) M is invertible, with the inverse matrix  $M^{-1} = \begin{pmatrix} 1 & 2 \\ 3 & 3 \end{pmatrix}$ .
- (2) The transposition transformation  $\pi$  corresponds to lane position transformation matrix  $\begin{pmatrix} 1 & 2 \\ 3 & 3 \end{pmatrix}$ .
- (3) The order of matrix M is 24, i.e.,  $\begin{pmatrix} 4 & 4 \\ 1 & 3 \end{pmatrix}^{24} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ . 24 is the maximum order of  $2 \times 2$  invertible matrices with elements in GF(5).
- (4) Any two points in the same row, after  $\pi$  transformation, cannot end up in the same row or same column. That is, for any two points with the same horizontal coordinate but different vertical coordinate, after  $\pi$  transformation, both horizontal and vertical coordinate are different. This property is easily verified: assume two arbitrary points with same horizontal coordinate but

different vertical coordinate are  $(u,v)$  and  $(u,v')$ ; after  $\pi$  transformation, the coordinate become  $(4u+4v, u+3v)$  and  $(4u+4v', u+3v')$ , from which it follows that  $4u+4v \neq 4u+4v'$  and  $u+3v \neq u+3v'$ .

(5) Any two points in the same column, after  $\pi$  transformation, cannot end up in the same row or same column. That is, for any two points with different horizontal coordinate but same vertical coordinate, after  $\pi$  transformation, both horizontal and vertical coordinate are different. This property is easily verified: assume two arbitrary points with different horizontal coordinate but same vertical coordinate are  $(u,v)$  and  $(u',v)$ ; after  $\pi$  transformation, the coordinate become  $(4u+4v, u+3v)$  and  $(4u'+4v, u'+3v)$ , from which it follows that  $4u+4v \neq 4u'+4v$  and  $u+3v \neq u'+3v$ .

### 3.2.4 Design of Constant Addition

Round constants are introduced in each round primarily to resist slide attacks, rotational attacks, and the like. The constants are taken from the fractional part of the natural constant  $e$ , demonstrating that the constant selection contains no hidden backdoors. The round constants are updated by multiplication in the finite field  $GF(2^{64})$  defined by the irreducible polynomial  $x^{64}+x^{63}+x^{62}+x^{54}+x^{40}+x^{35}+x^{15}+x+1$ , which is different from the operations of the linear transformation  $\theta$  (performed in the ring  $GF(2)[x]/(x^{64}+1)$ ), thereby avoiding related attacks that could arise from combining the two.

Round constants use only 64 bits per round, balancing implementation efficiency and resource consumption.

## 3.3 Parameter Selection Rationale

We selected the corresponding parameters based on the properties required for the various cryptographic components mentioned above, and in conjunction with security analysis, further screened from the parameters that satisfy these properties those with strongest resistance against differential, linear, and other attacks.

## 4 Security Claims and Analysis

### 4.1 Theoretical Security

#### 4.1.1 Security Analysis of the Mode of Operation

Assume that in the mode of operation of the Litchi algorithm, the capacity is  $c$ , the hash output length is  $|h|$  bits,  $l$  is the number of blocks in the second-preimage challenge message, and  $t$  is the bit length of the counter. Then, the complexity of resisting various attacks for Litchi is as follows:

- (1) Collision attack:  $\min(O(2^{|h|/2}), O(2^{c/2}))$ ;
- (2) Preimage attack:  $\min(O(2^{|h|}), O(2^c))$ ;
- (3) Second preimage attack:  $\min(O(2^{|h|}), O(2^{c-\log_2 l}))$ ;
- (4) Quantum collision attack:  $\min(O(2^{|h|/4}), O(2^{c/4}))$ ;
- (5) Quantum preimage attack:  $\min(O(2^{|h|/2}), O(2^{c/2}))$ ;
- (6) Quantum second preimage attack:  $\min(O(2^{|h|/2}), O(2^{(c-\log_2 l)/2}))$ ;

(7) If the permutation  $P$  is a random permutation, even if the adversary can query the permutation arbitrarily, the complexity for distinguishing the Litchi algorithm from a random oracle is  $\min(O(2^{(c-t)/2}), 2^{1600-\min(c, |h|)})$ . Among them, when the Litchi mode of operation is used as a hash function, the output consists of only one block  $h_1$ , and the counter value can be regarded as a constant, so  $t=0$ ; when used as an XOF, the output length is less than  $2^{64}$  bits, so  $t \leq 64$ .

When the Litchi mode of operation is used as a fixed-output-length hash function, it is completely identical to the  $\text{Sponge-F}^{\text{P,pd}}$  construction. Therefore, the proof of the above conclusions can be found in [GHJ+25].

There is a significant difference between the Litchi mode of operation and the  $\text{Sponge-F}^{\text{P,pd}}$  construction when used as an XOF: the Litchi mode adopts a counter-based parallel method, whereas the  $\text{Sponge-F}^{\text{P,pd}}$  construction uses a serial method. Hence, we specifically analyze the security of Litchi when used as an XOF.

When the Litchi mode of operation is used as an XOF, we have  $h = \text{MSB}_{|h|}(h_1 || h_2 || \dots || h_{\lceil |h|/c \rceil})$ . To find a preimage of  $h$ , an adversary must find a preimage of  $h_1$ . For example, to find a preimage of  $h$ , one must find a preimage of  $h_1$ . Because of the use of counter mode, the inputs to  $P$  are all distinct, and  $P$  is a random permutation. Thus, each  $h_i$  is independently random; obtaining the values of other  $h_i$  reveals no information about  $h_1$ .

When processing the last message block in Litchi, let  $X = S_{l-1} \oplus (M_{l-1} || 0^c)$ , and the  $i$ -th output block  $h_i$  can be viewed as a function of  $X$ , denoted  $h_i = f_i(X)$ , where  $f_i(X) = \text{LSB}_c(P(X \oplus (0^r || [i]_c)) \oplus (X \oplus (0^r || [i]_c)))$ ,  $i = 1, 2, \dots, s = \lceil |h|/c \rceil$ . Therefore, for a fixed  $X$ , in the ideal permutation model, these  $h_i$  are independent random values, and all output blocks  $h_i$  can be directly computed from  $X$ .

Moreover, according to the design of Litchi, the concrete value of  $X$  is fully determined by the message  $M$ , denoted  $X = g(M)$ . Hence,  $h_i$  can also be written as  $h_i = f_i(g(M))$ ,  $i = 1, 2, \dots, s = \lceil |h|/c \rceil$ .

Based on the above analysis, if Litchi is used solely as a fixed-output-length

hash algorithm (with only one output block), the output block is  $h_1 = f_1(g(M))$ . If Litchi is used as an XOF to output  $s$  blocks, then Litchi-XOF is equivalent to independently executing  $s$  fixed-output-length Litchi algorithms (i.e., Litchi-XOF consists of  $s$  independent algorithms  $f_i(g(M))$ ), each using a different  $i$  for separation. In the ideal permutation model, this design achieves perfect independence. This property makes attacking Litchi-XOF more difficult than attacking a fixed-output-length hash algorithm. Below, we analyze the security of Litchi-XOF in comparison to Litchi hash algorithms with respect to preimage resistance, collision resistance, second preimage resistance, and indistinguishability.

### (1) Preimage Resistance

Intuitively, given  $s$  independent output blocks  $h_1 h_2 \dots h_s$  of Litchi-XOF, finding an  $M$  such that  $f_i(g(M)) = h_i (i = 1, 2, \dots, s)$  simultaneously is harder than finding an  $M$  that satisfies only one equation  $f_1(g(M)) = h_1$ . Conversely, the reverse does not necessarily hold. Therefore, the preimage resistance of Litchi-XOF should be stronger than that of the Litchi hash algorithm. A rigorous analysis follows.

Given the output blocks  $h_1 h_2 \dots h_s$  of Litchi-XOF, if there exists a preimage attacker  $\mathcal{A}$  making  $q$  queries that outputs a preimage  $M$  such that  $f_i(g(M)) = h_i$  for all  $i = 1, 2, \dots, s$ , then we can construct an attacker  $\mathcal{B}$  as follows: first invoke  $\mathcal{A}$  to obtain  $M$  (where  $M$  is assumed to have  $l$  message blocks), then compute  $g(M)$  to obtain an  $X$  such that  $f_i(X) = h_i$  for all  $i$ . That is,  $\mathcal{B}$  solves the following hard problem **HP**: find  $X$  such that  $f_i(X) = h_i$ , for all  $i = 1, 2, \dots, s$ . The running time of  $\mathcal{B}$  is essentially the same as that of  $\mathcal{A}$ , and the number of queries is  $q + l$ . Hence, the preimage advantage of  $\mathcal{A}$ ,  $\text{Adv}_{\text{XOF}}^{\text{pre}}(\mathcal{A}, q)$ , does not exceed the advantage of  $\mathcal{B}$  in solving **HP**, denoted  $\text{Adv}^{\text{HP}}(\mathcal{B}, q + l)$ .

In the ideal permutation model, after 1 query, the probability that  $\mathcal{B}$  finds an  $X$  satisfying  $f_i(X) = h_i$  for a single  $i$  is at most  $\frac{2^{b-c}}{2^{b-q-l}}$ . Since the  $s$  relations are completely independent, the probability of finding an  $X$  that simultaneously satisfies all  $s$  relations is at most  $\left(\frac{2^{b-c}}{2^{b-q-l}}\right)^s$ . Taking the union bound over all  $q + l$  queries, we obtain:  $\text{Adv}^{\text{HP}}(\mathcal{B}, q + l) \leq (q + l) \cdot \left(\frac{2^{b-c}}{2^{b-q-l}}\right)^s$ . Consequently,

$$\text{Adv}_{\text{XOF}}^{\text{pre}}(\mathcal{A}, q) \leq \text{Adv}^{\text{HP}}(\mathcal{B}, q + l) \leq (q + l) / \left(2^c - \frac{q + l}{2^{b-c}}\right)^s$$

For the Litchi hash algorithm, the upper bound on preimage advantage is  $\text{Adv}_{\text{HASH}}^{\text{pre}}(\mathcal{A}, q) \leq (q + l) / \left(2^c - \frac{q + l}{2^{b-c}}\right)$ , which corresponds to the case  $s=1$  in

the above formula. Therefore, the preimage resistance of Litchi-XOF is no lower than the Litchi hash algorithm.

## (2) Collision Resistance

Assume that the second-preimage resistance of Litchi-XOF is lower than that of the Litchi hash algorithm. Then, an adversary could obtain a second preimage of the Litchi hash algorithm for message  $M$  by finding a second preimage of Litchi-XOF for  $M$ . The specific method is as follows :

Given a message  $M$ , compute  $h_1||h_2||\dots||h_s = \text{Litchi-XOF}(M)$ . Find  $M'$  such that  $M' \neq M$  and  $\text{Litchi-XOF}(M) = \text{Litchi-XOF}(M')$ . Then  $M'$  is also a second preimage of  $M$  when Litchi is used as a hash algorithm. Thus, a second preimage for the Litchi hash algorithm can be obtained at the cost of computing a second preimage for Litchi-XOF. This contradicts the assumption that the second-preimage resistance of Litchi-XOF is lower than that of the Litchi hash algorithm. Therefore, the second-preimage resistance of Litchi-XOF is no lower than that of the Litchi hash algorithm.

## (3) Second Preimage Resistance

If a collision pair  $M_1$  and  $M_2$  satisfying  $\text{Litchi-XOF}(M_1) = \text{Litchi-XOF}(M_2)$  is found, then  $M_1$  and  $M_2$  must also be a collision pair for Litchi when used as a hash function. Therefore, the collision resistance of Litchi-XOF is no lower than that of the Litchi hash algorithm.

## (4) Indifferentiability

By treating the counter in the Litchi-XOF structure as  $t$ -bit message, the security capacity becomes  $c-t$ . According to the indifferentiability result of [GHJ+25], the indifferentiability security bound of the Litchi-XOF structure is at least  $\min(O(2^{(c-t)/2}), 2^{1600-\min(c, |h|)})$ .

### 4.1.2 Analysis of the Permutation P

#### 4.1.2.1 Differential Analysis

Differential analysis is the most important analysis for permutation P, and it is also a crucial basis for our parameter selection of the linear transformation  $\theta_i$  and the design of the transposition transformation  $\pi$ .

Since the constant addition  $\iota$  does not need to be considered in differential analysis, permutation P can be analyzed by considering only the round function  $f$  composed of  $\pi \circ \theta \circ \chi$ . For the  $i$ -th round, let the input difference of the  $\chi$  transformation be  $\Delta X_i$ , the input difference of the  $\theta$  transformation be  $\Delta Y_i$  (i.e., the output difference of  $\chi$ ), and the input difference of the  $\pi$  transformation be

$\Delta Z_i$  (i.e., the output difference of  $\theta$ ). That is:

$$\Delta X_i \xrightarrow{\chi} \Delta Y_i \xrightarrow{\theta} \Delta Z_i \xrightarrow{\pi} \Delta X_{i+1}.$$

### (1) 2-Round Differential Analysis

Since all  $\theta_i$  transformations have a differential branch number of 12, and after the  $\pi$  transformation any two lanes from the same plane are not in the same column, it is easy to prove that the minimum number of active S-boxes for a 2-round differential of permutation P is 12. Through computer search, it can similarly be obtained that the minimum number of active S-boxes for a 2-round differential of permutation P is 12.

### (2) 3-Round Differential Analysis

For 3-round differential analysis, we use an MILP approach to search for active S-boxes. The S-box is characterized by its difference distribution table, and the linear transformation is characterized by bitwise XOR. Direct 3-round search is very inefficient; we use certain techniques to search for lower and upper bounds on the minimum number of 3-round differential active S-boxes.

We primarily obtain a lower bound for the 3-round minimum differential active S-boxes through a detailed 2-round differential active S-box search. Specifically, we search for the minimum number of 2-round differential active S-boxes with input  $\Delta Y_i$  and output  $\Delta X_{i+1}$ :

$$\Delta Y_i \xrightarrow{\theta} \Delta Z_i \xrightarrow{\pi} \Delta X_{i+1}$$

For convenience, denote  $\Delta Y_{i,j}$  as the  $j$ -th plane of  $\Delta Y_i$ . We consider 32 cases based on whether  $\Delta Y_{i,j}$  is zero. For these 32 cases, we compute the number of active S-boxes for 2 rounds, as shown in Table 4-1, where 0 indicates  $\Delta Y_{i,j}=0$  and 1 indicates  $\Delta Y_{i,j}\neq 0$ . Due to high complexity, when the number of non-zero input difference planes in  $\Delta Y_i$  exceeds 3, we cannot obtain the minimum number of 2-round differential active S-boxes; using Theorem 4-1, we provide a conservative lower bound.

Table 4-1 Detailed Values of 2-Round Differential Active S-Boxes

| $\Delta Y_{i,0}, \Delta Y_{i,1}, \Delta Y_{i,2},$<br>$\Delta Y_{i,3}, \Delta Y_{i,4}$ | Active S-boxes | $\Delta Y_{i,0}, \Delta Y_{i,1}, \Delta Y_{i,2},$<br>$\Delta Y_{i,3}, \Delta Y_{i,4}$ | Active S-boxes |
|---------------------------------------------------------------------------------------|----------------|---------------------------------------------------------------------------------------|----------------|
| 0,0,0,0,0                                                                             | 0              | 0,0,1,1,1                                                                             | 23             |
| 0,0,0,0,1                                                                             | 12             | 0,1,0,1,1                                                                             | 24             |
| 0,0,0,1,0                                                                             | 12             | 0,1,1,0,1                                                                             | 23             |
| 0,0,1,0,0                                                                             | 12             | 0,1,1,1,0                                                                             | 23             |
| 0,1,0,0,0                                                                             | 12             | 1,0,0,1,1                                                                             | 23             |
| 1,0,0,0,0                                                                             | 12             | 1,0,1,0,1                                                                             | 23             |

|           |    |           |           |
|-----------|----|-----------|-----------|
| 0,0,0,1,1 | 18 | 1,0,1,1,0 | 23        |
| 0,0,1,0,1 | 18 | 1,1,0,0,1 | 24        |
| 1,0,0,1,0 | 18 | 1,1,0,1,0 | 24        |
| 0,1,0,0,1 | 18 | 1,1,1,0,0 | 24        |
| 0,1,0,1,0 | 18 | 0,1,1,1,1 | $\geq 24$ |
| 0,1,1,0,0 | 18 | 1,0,1,1,1 | $\geq 24$ |
| 1,0,0,0,1 | 18 | 1,1,0,1,1 | $\geq 24$ |
| 0,0,1,1,0 | 18 | 1,1,1,0,1 | $\geq 24$ |
| 1,0,1,0,0 | 18 | 1,1,1,1,0 | $\geq 24$ |
| 1,1,0,0,0 | 18 | 1,1,1,1,1 | $\geq 24$ |

From Table 4-1, it can be seen that when the number of non-zero input difference planes in  $Y_i$  exceeds 2, the minimum value of 2-round differential active S-boxes is 23.

**Theorem 4-1:** Assuming  $\Delta Y_i$  originally has  $t$  planes with non-zero input difference, then after adding one more plane with non-zero input difference, the minimum number of 2-round differential active S-boxes will not decrease.

*Proof:* Assume  $\Delta Y_i$  with one additional plane's input difference added is  $\Delta Y'_i$ , and let  $\Delta X_{i+1} = \pi(\theta(Y_i))$ ,  $\Delta X'_{i+1} = \pi(\theta(Y'_i))$ . Since  $\Delta Y'_i$  has one additional plane's input differential mask compared to  $\Delta Y_i$ , according to the properties of  $\pi$  and  $\theta$ ,  $\Delta X'_{i+1}$  has additional non-zero lanes compared to  $\Delta X_{i+1}$ . Therefore, the number of active S-boxes for  $\Delta Y'_i$  is no less than that for  $\Delta Y_i$ , and the number of active S-boxes for  $\Delta X'_{i+1}$  is no less than that for  $\Delta X_{i+1}$ . Thus, the minimum number of 2-round differential active S-boxes will not decrease. ■

We now prove that when the number of non-zero input difference planes in round  $i$  is less than 3, the minimum number of differential active S-boxes over rounds  $i$ ,  $i+1$ , and  $i+2$  is at least 24. Since direct search is too time-consuming, we perform the following 2-round search:

$$\Delta Y_i \xrightarrow{\theta} \Delta Z_i \xrightarrow{\pi} \Delta X_{i+1} \xrightarrow{\chi} \Delta Y_{i+1}$$

We search for differential pairs with input  $\Delta Y_i$  and output  $\Delta Y_{i+1}$ . To obtain refined search results, we exhaustively search all possible values where the plane weights of  $\Delta Y_i$  and  $\Delta Y_{i+1}$  are 1 and 2 (we define the plane weight of  $\Delta Y_i$  as the number of planes with non-zero differential in  $\Delta Y_i$ ), totaling  $15 \times 15 = 225$  cases. It can be computed that the minimum active S-boxes for these 225 cases are all no less than 23. In fact, due to time constraints, we stopped searching when the lower bound reached 23; actual values may exceed 23. When a real differential path exists, the minimum number of differential active S-boxes we searched for is 26, at which point the plane weight of  $\Delta Y_i$  is 1 and the plane weight of  $\Delta Y_{i+1}$  is 2.

From the above analysis, when the plane weight of  $\Delta Y_i$  is at most 2 and that of  $\Delta Y_{i+1}$  is at most 2, the number of active S-boxes over three consecutive rounds is at least 24.

In summary, considering the following three cases, the minimum number of active S-boxes over three consecutive rounds of permutation P is no less than 24.

1) When the plane weight of  $\Delta Y_i$  is greater than 2, from Table 4-1, the number of active S-boxes in rounds  $i$  and  $i+1$  is no less than 23, so over three rounds it is no less than 24.

2) When the plane weight of  $\Delta Y_i$  is at most 2 and the plane weight of  $\Delta Y_{i+1}$  is greater than 2, from Table 4-1, the number of active S-boxes in rounds  $i+1$  and  $i+2$  is no less than 23, so over three rounds it is no less than 24.

3) When the plane weights of  $\Delta Y_i$  and  $\Delta Y_{i+1}$  are both at most 2, then the number of active S-boxes over three consecutive rounds is at least 24.

In fact, for 3-round differential active S-boxes, the calculation of the above 24 active S-boxes is very conservative. We performed an MILP search over rounds  $i, i+1, i+2$ . After many attempts and prolonged searching, we found that when  $\Delta Y_i$  has only one plane with non-zero input difference and the input and output differences of that plane are both 6, the minimum number of active S-boxes for a real 3-round path is 64, i.e., the upper bound for the number of 3-round differential active S-boxes is 64, as shown in Table 4-2. **We conjecture that 64 is very likely the lower bound for the number of 3-round differential active S-boxes.**

Table 4-2: Detailed Values of 3-Round Differential Active S-Boxes

| $\Delta Y_{i,0}, \Delta Y_{i,1}, \Delta Y_{i,2}, \Delta Y_{i,3}, \Delta Y_{i,4}$ | Active S-boxes | Active S-boxes per Round |
|----------------------------------------------------------------------------------|----------------|--------------------------|
| 1,0,0,0,0                                                                        | 65             | 6---6---53               |
| 0,1,0,0,0                                                                        | 66             | 6---6---54               |
| 0,0,1,0,0                                                                        | 66             | 6---6---54               |
| 0,0,0,1,0                                                                        | 64             | 6---6---52               |
| 0,0,0,0,1                                                                        | 69             | 6---6---57               |

In summary, the number of differential active S-boxes for permutation P is shown in Table 4-3.



Table 4-3: Number of Differential Active S-boxes for Permutation P

| Rounds                          | 1 | 2  | 3       |
|---------------------------------|---|----|---------|
| Min Differential Active S-boxes | 1 | 12 | [24,64] |

#### 4.1.2.2 Linear Analysis

Since the constant addition is not needed to be considered in linear analysis, permutation P can be analyzed by considering only the round function  $f$  composed of  $\pi \circ \theta \circ \chi$ . For linear masks, the computation can be expressed as  $\Gamma X_{i+1} = \theta^T(\pi^T(\chi^{-1}(\Gamma X_i)))$ . From Sections 3.2.2 and 3.2.3, we have  $\theta^T = \theta$  and  $\pi^T = \pi^{-1}$ , so  $\Gamma X_{i+1} = \theta(\pi^{-1}(\chi^{-1}(\Gamma X_i)))$ . For the  $i$ -th round, let the input mask of the  $\chi^{-1}$  transformation be  $\Gamma X_i$ , the input mask of the  $\pi^{-1}$  transformation be  $\Gamma Z_i$  (i.e., the output mask of  $\chi^{-1}$ ), and the input mask of the  $\theta$  transformation be  $\Gamma Y_i$  (i.e., the output mask of  $\pi^{-1}$ ). That is:

$$\Gamma X_i \xrightarrow{\chi^{-1}} \Gamma Z_i \xrightarrow{\pi^{-1}} \Gamma Y_i \xrightarrow{\theta} \Gamma X_{i+1}$$

##### (1) 2-Round Linear Analysis

Since all  $\theta_i$  transformations have linear branch number 12, and after  $\pi^{-1}$  transformation, any two lanes from the same plane are not in the same column, it is easy to prove that the minimum number of active S-boxes for 2-round linear analysis of permutation P is 12. Through computer search, it can similarly be obtained that the minimum number of active S-boxes for 2-round linear analysis of permutation P is 12.

##### (2) 3-Round Linear Analysis

For 3-round linear analysis, we use MILP methods to search for linear active S-boxes. The S-box is characterized using its linear approximation table and converted to inequalities; linear transformation is characterized using bitwise XOR. Direct 3-round search is very inefficient; we use certain techniques to search for lower and upper bounds on the minimum number of 3-round linear active S-boxes.

We primarily obtain the lower bound for 3-round minimum linear active S-boxes through detailed 2-round linear active S-box analysis. Specifically, we search for the minimum number of 2-round linear active S-boxes with input  $\Gamma Z_i$  and output  $\Gamma X_{i+1}$ :

$$\Gamma Z_i \xrightarrow{\pi^{-1}} \Gamma Y_i \xrightarrow{\theta} \Gamma X_{i+1}$$

For convenience of expression, we denote  $\Gamma Y_{i,j}$  as the  $j$ -th plane of  $\Gamma Y_i$ . We divide into 32 cases based on whether  $\Gamma Y_{i,j}$  is zero. Then for these 32 cases, we separately compute the number of active S-boxes for 2 rounds, as shown in

Table 4-4, where 0 indicates  $\Gamma Y_{ij}=0$ , and 1 indicates  $\Gamma Y_{ij}\neq 0$ . Due to excessive complexity, when the number of non-zero input linear mask planes in  $Y_i$  exceeds 3, we cannot compute the minimum number of worst-case linear active S-boxes for 2 rounds; using Theorem 4-2, we provide conservative lower bound values for active S-boxes.

Table 4-4: Detailed Values of 2-Round Linear Active S-Boxes

| $\Gamma Y_{i,0}, \Gamma Y_{i,1}, \Gamma Y_{i,2},$<br>$\Gamma Y_{i,3}, \Gamma Y_{i,4}$ | Active S-boxes | $\Gamma Y_{i,0}, \Gamma Y_{i,1}, \Gamma Y_{i,2},$<br>$\Gamma Y_{i,3}, \Gamma Y_{i,4}$ | Active S-boxes |
|---------------------------------------------------------------------------------------|----------------|---------------------------------------------------------------------------------------|----------------|
| 0,0,0,0,0                                                                             | 0              | 0,0,1,1,1                                                                             | 23             |
| 0,0,0,0,1                                                                             | 12             | 0,1,0,1,1                                                                             | 24             |
| 0,0,0,1,0                                                                             | 12             | 0,1,1,0,1                                                                             | 23             |
| 0,0,1,0,0                                                                             | 12             | 0,1,1,1,0                                                                             | 23             |
| 0,1,0,0,0                                                                             | 12             | 1,0,0,1,1                                                                             | 23             |
| 1,0,0,0,0                                                                             | 12             | 1,0,1,0,1                                                                             | 23             |
| 0,0,0,1,1                                                                             | 18             | 1,0,1,1,0                                                                             | 23             |
| 0,0,1,0,1                                                                             | 18             | 1,1,0,0,1                                                                             | 23             |
| 1,0,0,1,0                                                                             | 18             | 1,1,0,1,0                                                                             | 23             |
| 0,1,0,0,1                                                                             | 18             | 1,1,1,0,0                                                                             | 24             |
| 0,1,0,1,0                                                                             | 18             | 0,1,1,1,1                                                                             | $\geq 24$      |
| 0,1,1,0,0                                                                             | 18             | 1,0,1,1,1                                                                             | $\geq 23$      |
| 1,0,0,0,1                                                                             | 18             | 1,1,0,1,1                                                                             | $\geq 24$      |
| 0,0,1,1,0                                                                             | 18             | 1,1,1,0,1                                                                             | $\geq 24$      |
| 1,0,1,0,0                                                                             | 18             | 1,1,1,1,0                                                                             | $\geq 24$      |
| 1,1,0,0,0                                                                             | 18             | 1,1,1,1,1                                                                             | $\geq 24$      |

From Table 4-4, it can be seen that when the number of non-zero input linear mask planes in  $\Gamma Y_i$  exceeds 2, the minimum value of 2-round linear active S-boxes is 23.

**Theorem 4-2:** Assuming  $\Gamma Y_i$  originally has  $t$  planes with non-zero input linear mask, then after adding one more plane with non-zero input linear mask, the minimum number of 2-round linear active S-boxes will not decrease.

*Proof:* Assume  $\Gamma Y_i$  with one additional plane's input linear mask added is  $\Gamma Y'_i$ , and let  $\Gamma Z_i = \pi(\Gamma Y_i)$ ,  $\Gamma X_{i+1} = \theta(\Gamma Y_i)$ ;  $\Gamma Z'_i = \pi(\Gamma Y'_i)$ ,  $\Gamma X'_{i+1} = \theta(\Gamma Y'_i)$ . Since  $\Gamma Y'_i$

has one additional plane's input linear mask compared to  $\Gamma Y_i$ , according to the properties of  $\pi$ ,  $\Gamma Z'_i$  has additional non-zero lanes compared to  $\Gamma Z_i$ , and  $\Gamma X'_{i+1}$  has additional non-zero lanes compared to  $\Gamma X_{i+1}$ . Therefore, the number of active S-boxes for  $\Gamma Z'_i$  is no less than that for  $\Gamma Z_i$ , and the number of active S-boxes for  $\Gamma X'_{i+1}$  is no less than that for  $\Gamma X_{i+1}$ . Thus, the minimum number of 2-round linear active S-boxes will not decrease. ■

We now prove that when the number of non-zero input linear mask planes in round  $i$  is less than 3, the minimum number of linear active S-boxes over rounds  $i$ ,  $i+1$ , and  $i+2$  is at least 24. Since direct search is too time-consuming, we perform the following 2-round search:

$$\Gamma Y_i \xrightarrow{\theta} \Gamma X_{i+1} \xrightarrow{\chi^{-1}} \Gamma Z_{i+1} \xrightarrow{\pi^{-1}} \Gamma Y_{i+1}$$

We search for linear mask pairs with input  $\Gamma Y_i$  and output  $\Gamma Y_{i+1}$ . To obtain fine-grained results, we exhaustively search over all possibilities where the plane weights of  $\Gamma Y_i$  and  $\Gamma Y_{i+1}$  are 1 or 2, totaling  $15 \times 15 = 225$  cases. In the search process, we set the lower bound to stop at 23. The results show that for 224 cases the minimum linear active S-boxes are no less than 23. Only for the case where  $\Gamma Y_{i,0}||\Gamma Y_{i,1}||\Gamma Y_{i,2}||\Gamma Y_{i,3}||\Gamma Y_{i,4}=11000$  and  $\Gamma Y_{i+1,0}||\Gamma Y_{i+1,1}||\Gamma Y_{i+1,2}||\Gamma Y_{i+1,3}$

$||\Gamma Y_{i+1,4}=11000$ , the searched minimum linear active S-boxes is 20. In this case, the number of linear active S-boxes for  $\Gamma Y_i$  is no less than 10, and since the plane weight of  $\Gamma Y_{i+1}$  is 2, from Table 4-4 the number of linear active S-boxes in rounds  $i+1$  and  $i+2$  is no less than 18. Therefore, the total number over three rounds is no less than  $10+18=28$ .

From the above analysis, when the plane weight of  $\Gamma Y_i$  is at most 2 and that of  $\Gamma Y_{i+1}$  is at most 2, the number of linear active S-boxes over three consecutive rounds is at least 24.

In summary, considering the following three cases, the minimum number of linear active S-boxes over three consecutive rounds of permutation P is no less than 24.

1)When the plane weight of  $\Gamma Y_i$  is greater than 2, from Table 4-4, the number of active S-boxes in rounds  $i$  and  $i+1$  is no less than 23, so over three rounds it is no less than 24.

2)When the plane weight of  $\Gamma Y_i$  is at most 2 and the plane weight of  $\Gamma Y_{i+1}$  is greater than 2, from Table 4-4, the number of active S-boxes in rounds  $i+1$  and  $i+2$  is no less than 23, so over three rounds it is no less than 24.

3)When the plane weights of  $\Gamma Y_i$  and  $\Gamma Y_{i+1}$  are both at most 2, then the number of linear active S-boxes over three consecutive rounds is at least 24.

In fact, for the number of 3-round linear active S-boxes, the calculation of the

above 24 active S-boxes is very conservative. We performed MILP search on active S-boxes for a total of 3 rounds  $i$ ,  $i+1$ , and  $i+2$ . After multiple attempts and extended searching, we found that when  $Y_i$  has only one plane with non-zero input mask and the input and output masks of that plane are both 6, the minimum number of active S-boxes for a real 3-round path is 64, i.e., the upper bound for the number of 3-round linear active S-boxes is 64, as shown in Table 4-5. **We conjecture that 64 is very likely the lower bound for the number of 3-round linear active S-boxes.**

Table 4-5: Detailed Values of 3-Round Linear Active S-Boxes

| $\Gamma Y_{i,0}, \Gamma Y_{i,1}, \Gamma Y_{i,2}, \Gamma Y_{i,3}, \Gamma Y_{i,4}$ | Active S-boxes | Active S-boxes per Round |
|----------------------------------------------------------------------------------|----------------|--------------------------|
| 1,0,0,0,0                                                                        | 64             | 6---6---52               |
| 0,1,0,0,0                                                                        | 64             | 6---6---52               |
| 0,0,1,0,0                                                                        | 67             | 6---6---54               |
| 0,0,0,1,0                                                                        | 68             | 6---6---56               |
| 0,0,0,0,1                                                                        | 66             | 6---6---54               |

In summary, the linear active S-boxes for permutation P is shown in Table 4-6.

Table 4-6: Linear Active S-Box Count for Permutation P

| Rounds                    | 1 | 2  | 3       |
|---------------------------|---|----|---------|
| Min Linear Active S-boxes | 1 | 12 | [24,64] |

#### 4.1.2.3 Integral Analysis

The overall structure of permutation P is an SPN with width 1600 bits, using 5-bit S-boxes of algebraic degree 3; therefore, permutation P is a (5,3,320)-SPN. Todo [Tod15] provided integral distinguisher results for (5,2,320)-SPN and (5,4,320)-SPN, but did not give results for (5,3,320)-SPN. Using Algorithm 2 proposed by Todo in [Tod15], we obtained integral distinguisher results for permutation P (i.e., (5,3,320)-SPN) as shown in Table 4-7.

Table 4-7: Integral Distinguishers for Permutation P

| Rounds          | 5        | 6         | 7         | 8          | 9          | 10         | 11         |
|-----------------|----------|-----------|-----------|------------|------------|------------|------------|
| Data Complexity | $2^{84}$ | $2^{245}$ | $2^{730}$ | $2^{1310}$ | $2^{1505}$ | $2^{1570}$ | $2^{1590}$ |

From Table 4-7, it can be seen that the integral distinguisher for permutation P has 11 rounds, with data complexity  $2^{1590}$ . Considering the actual security of Litchi algorithm ( $2^{1024}$ ), the distinguisher rounds can only reach 7. If more detailed characterization of S-boxes and linear transformation is

performed, integral distinguishers with more rounds might be obtained; however, considering the number of rounds of permutation P and actual security, Litchi algorithm can resist integral attacks

#### 4.1.2.4 Impossible Differential Analysis

$$X_0 \xrightarrow{\chi} Y_0 \xrightarrow{\theta} Z_0 \xrightarrow{\pi} X_1 \xrightarrow{\chi} \dots X_{r-1} \xrightarrow{\chi} Y_{r-1} \xrightarrow{\theta} Z_{r-1} \xrightarrow{\pi} X_r$$

For any  $r$ -round differential path above, since  $Y_i$  and  $X_{i+1}$  have a one-to-one correspondence, for efficient search, we only need to find one  $r$ -round impossible differential path from  $X_0$  to  $Y_{r-1}$  to find an  $r$ -round impossible differential path from  $X_0$  to  $X_r$ .

Through computer search, we found a 4-round impossible differential path with input difference  $X_0=1||0^{1599}$  and output  $Y_3=1||0^{1599}$ . The value of  $X_4$  at this point is shown in Table 4-8.

Table 4-8:  $X_4$  Values for Impossible Differential Path

|                                                                                           |
|-------------------------------------------------------------------------------------------|
| 8000000140000000, 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000020040000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, 0000040040000000, |
| 0000000000000000, 0800000040000000, 0000000000000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000000000000000, 4000000040000000, 0000000000000000. |

Due to computational complexity, we did not exhaust all differential paths, so longer-round impossible differential paths may exist. However, since the algorithm has sufficient round redundancy, we believe permutation P can resist impossible differential attacks.

#### 4.1.2.5 Zero-Correlation Linear Attack

$$X_0 \xrightarrow{\chi^{-1}} Z_0 \xrightarrow{\pi^{-1}} Y_0 \xrightarrow{\theta} X_1 \xrightarrow{\chi^{-1}} \dots X_{r-1} \xrightarrow{\chi^{-1}} Z_{r-1} \xrightarrow{\pi^{-1}} Y_{r-1} \xrightarrow{\theta} X_r$$

For any  $r$ -round linear path above, since  $Y_i$  and  $X_{i+1}$  have a one-to-one correspondence, for efficient search, we only need to find one  $r$ -round zero-correlation path from  $X_0$  to  $Y_{r-1}$  to find an  $r$ -round zero-correlation path from  $X_0$  to  $X_r$ .

Through computer search, we found a 4-round zero-correlation path with  $X_0=1||0^{1599}$  and output  $Y_3=1||0^{1599}$ . The value of  $X_4$  at this point is shown in Table 4-9.

Due to computational complexity, we did not exhaust all zero-correlation paths, so longer-round zero-correlation paths may exist. However, since the algorithm has sufficient round redundancy, we believe permutation P can resist zero-correlation attacks.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 80000000140000000, 40000000400000000, 08000000400000000, 00000400400000000, 00000200400000000,<br>00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000,<br>00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000,<br>00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000,<br>00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000。 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

The low algebraic degree of the S-box allows the construction of internal and external zero-sum distinguishers on the permutation, preventing it from being treated as an ideal random permutation. The algebraic degree of the round function  $f$  of permutation  $P$  is 3. The upper bound on algebraic degree after  $R$  rounds is shown in Table 4-10 (estimated via Theorem 2 of [BC13]).

|                |      |      |      |      |      |      |      |      |
|----------------|------|------|------|------|------|------|------|------|
| Rounds R       | 1    | 2    | 3    | 4    | 5    | 6    | 7    | 8    |
| $\deg(f^R)$    | 3    | 9    | 27   | 81   | 243  | 729  | 1309 | 1503 |
| $\deg(f^{-R})$ | 3    | 9    | 27   | 81   | 243  | 729  | 1309 | 1503 |
| Rounds R       | 9    | 10   | 11   | 12   | 13   | 14   | 15   | 16   |
| $\deg(f^R)$    | 1567 | 1589 | 1596 | 1598 | 1599 | 1599 | 1599 | 1599 |
| $\deg(f^{-R})$ | 1567 | 1589 | 1596 | 1598 | 1599 | 1599 | 1599 | 1599 |

#### 4.1.2.7 Resistance Against Known Quantum Attacks

26

If considering the specific structure of permutation P, by utilizing its differential, linear, and other characteristics combined with Grover search and other quantum algorithms, quantum differential, linear, and other attacks can be performed, formulating quantum variants of these traditional analysis schemes. However, these quantum analysis methods merely replace the search portion of traditional analysis schemes with Grover's algorithm or QAA and other quantum algorithms; their attack strength compared to corresponding classical analysis schemes can achieve at most quadratic speedup. Since permutation P of Litchi algorithm was designed with full consideration of resistance against classical differential, linear, and other attack methods, under current analysis, the attack complexity of corresponding quantum attack schemes is higher than that of generic quantum attacks.

## 4.2 Concrete Security

The Litchi algorithm, denoted according to its output hash length as Litchi-512, Litchi-768, and Litchi-1024, has the following security results:

Table 4-11 Security Results of the Litchi Algorithm

| Algorithm   | Output Size | Classical Security Strengths in Bits |                 |                  | Quantum Security Strengths in Bits |                  |                       |
|-------------|-------------|--------------------------------------|-----------------|------------------|------------------------------------|------------------|-----------------------|
|             |             | Preimage                             | Second Preimage | Collision        | Preimage                           | Second Preimage  | Collision             |
| Litchi-512  | 512         | 512                                  | 512             | 256              | 256                                | 256              | $\geq 128$            |
| Litchi-768  | 768         | 768                                  | 768             | 384              | 384                                | 384              | $\geq 192$            |
| Litchi-1024 | 1024        | 1024                                 | 1024            | 512              | 512                                | 512              | $\geq 256$            |
| Litchi-XOF  | $d$         | $\min(d, 1024)$                      | $\min(d, 960)$  | $\min(d/2, 512)$ | $\min(d/2, 512)$                   | $\min(d/2, 480)$ | $\geq \min(d/4, 256)$ |

For quantum attacks, [GHJ+25] have proven that when the capacity is  $c$  and the hash output length is  $|h|$  bits, the quantum collision resistance of the Litchi mode of operation is at least  $\min(O(2^{|h|/4}), O(2^{c/4}))$ . The generic quantum collision attack complexity is  $O(2^{|h|/3})$ . The fact that our proven bound is lower than the generic quantum collision bound does not imply that our algorithm fails to achieve the generic quantum collision resistance; rather, it is due to the current proof techniques not yet yielding optimal security bounds for the mode of operation. For example, for the Sponge construction, [ACMT25] have proven quantum collision resistance is  $\min(O(2^{|h|/5}), O(2^{c/5}))$ , which is even lower. In fact, no practical attack better than the generic quantum collision attack has been found to date.

## 4.3 Other Security Properties

### 4.3.1 Statistical Test Analysis

Statistical randomness testing primarily examines the randomness of the digest stream. The tests were performed on the digest stream generated by the Litchi-1024 algorithm. The significance level for randomness testing was set to  $\alpha=0.01$ . The test items include: frequency test, serial test (also known as the poker test in some contexts), autocorrelation test, and runs test (5 items in total). Message samples were generated using five different methods: random message generation, counter generation, output feedback generation, low Hamming weight generation, and high Hamming weight generation. Under each message sample generation method,  $2^{20}$  groups of 512-bit message samples were produced, and the message digest of each sample was calculated and concatenated into a digest stream. Under each generation method, a total of  $2^{30}$  bits of digest stream were obtained, which were then divided into 1024 groups, each of length  $2^{20}$  bits. With the significance level  $\alpha=0.01$ , the number of passing groups for each randomness test should be no less than 1005 ( $\lceil 1024 \times (1 - \alpha - 3 \times (\alpha \times (1 - \alpha) / 1024)^{1/2}) \rceil = 1005$ ).

The test results below demonstrate that the Litchi algorithm passes the statistical randomness tests.

Table 4-12 Random Message Generation Method

| Test No. | Test Item       | Test Parameters | Test Groups | Passing Groups |
|----------|-----------------|-----------------|-------------|----------------|
| 1        | Frequency Test  | None            | 1024        | 1016           |
| 2        | Serial Test     | None            | 1024        | 1012           |
| 3        | Poker Test      | 4               | 1024        | 1012           |
|          |                 | 8               | 1024        | 1012           |
| 4        | Autocorrelation | 1               | 1024        | 1020           |
|          |                 | 2               | 1024        | 1012           |
|          |                 | 8               | 1024        | 1008           |
|          |                 | 16              | 1024        | 1012           |
| 5        | Runs Test       | None            | 1024        | 1020           |

Table 4-13 Counter Generation Method

| Test No. | Test Item       | Test Parameters | Test Groups | Passing Groups |
|----------|-----------------|-----------------|-------------|----------------|
| 1        | Frequency Test  | None            | 1024        | 1011           |
| 2        | Serial Test     | None            | 1024        | 1013           |
| 3        | Poker Test      | 4               | 1024        | 1013           |
|          |                 | 8               | 1024        | 1019           |
| 4        | Autocorrelation | 1               | 1024        | 1013           |



|   |           |      |      |      |
|---|-----------|------|------|------|
|   |           | 2    | 1024 | 1013 |
|   |           | 8    | 1024 | 1012 |
|   |           | 16   | 1024 | 1015 |
| 5 | Runs Test | None | 1024 | 1013 |

Table 4-14 Output Feedback Generation Method

| Test No. | Test Item       | Test Parameters | Test Groups | Passing Groups |
|----------|-----------------|-----------------|-------------|----------------|
| 1        | Frequency Test  | None            | 1024        | 1011           |
| 2        | Serial Test     | None            | 1024        | 1013           |
| 3        | Poker Test      | 4               | 1024        | 1010           |
|          |                 | 8               | 1024        | 1016           |
| 4        | Autocorrelation | 1               | 1024        | 1014           |
|          |                 | 2               | 1024        | 1016           |
|          |                 | 8               | 1024        | 1012           |
|          |                 | 16              | 1024        | 1015           |
| 5        | Runs Test       | None            | 1024        | 1014           |

Table 4-15 Low Hamming Weight Generation Method

| Test No. | Test Item       | Test Parameters | Test Groups | Passing Groups |
|----------|-----------------|-----------------|-------------|----------------|
| 1        | Frequency Test  | None            | 1024        | 1009           |
| 2        | Serial Test     | None            | 1024        | 1013           |
| 3        | Poker Test      | 4               | 1024        | 1014           |
|          |                 | 8               | 1024        | 1010           |
| 4        | Autocorrelation | 1               | 1024        | 1012           |
|          |                 | 2               | 1024        | 1012           |
|          |                 | 8               | 1024        | 1009           |
|          |                 | 16              | 1024        | 1013           |
| 5        | Runs Test       | None            | 1024        | 1012           |

Table 4-16 High Hamming Weight Generation Method

| Test No. | Test Item      | Test Parameters | Test Groups | Passing Groups |
|----------|----------------|-----------------|-------------|----------------|
| 1        | Frequency Test | None            | 1024        | 1019           |
| 2        | Serial Test    | None            | 1024        | 1021           |

|   |                 |      |      |      |
|---|-----------------|------|------|------|
| 3 | Poker Test      | 4    | 1024 | 1014 |
|   |                 | 8    | 1024 | 1013 |
| 4 | Autocorrelation | 1    | 1024 | 1018 |
|   |                 | 2    | 1024 | 1009 |
|   |                 | 8    | 1024 | 1017 |
|   |                 | 16   | 1024 | 1014 |
| 5 | Runs Test       | None | 1024 | 1018 |

### 4.3.2 Length Extension Attacks

A length extension attack means the following: for an unknown message  $M$ , given its hash value  $H(M)$ , does there exist a bit string  $x$  such that  $H(M||x)$  can be computed. This attack is effective against early hash algorithms based on the MD structure, because they often output the complete final chaining value. By appropriately adjusting the padding process and computing the compression function in sequence, a new hash value can be successfully constructed.

For the mode of operation used by the Litchi algorithm, as shown in Figure 2-1, the lower half of the state in the squeezing phase is XORed with a non-zero constant. Therefore, when the number of padded message blocks of  $M||x$  is different from that of  $M$ ,  $H(M)$  cannot be an intermediate state in the computation of  $H(M||x)$ . If the number of padded message blocks of  $M||x$  is the same as that of  $M$ , then using  $H(M)$  to compute  $H(M||x)$  would require inverting the final output phase of  $H(M)$ , which is infeasible.

In summary, the Litchi algorithm is resistant to length extension attacks.

## 5 Performance Evaluation

### 5.1 Performance Analysis

#### 5.1.1 Design Efficiency of the Overall Structure

##### (1) Software-Hardware Compatibility of the Mode of Operation

The mode of operation adopted by Litchi unifies message compression and hash value output into iterations of the same permutation function  $P$ , effectively improving implementation efficiency in both software and hardware.

- **Software level:** The cyclic iteration approach facilitates implementation as compact loop code; the state can be stored contiguously in registers or memory, reducing function call overhead. The symmetry between message compression and hash output phases reduces code branching complexity.
- **Hardware level:** A unified permutation function means that only one instance of the  $P$  circuit needs to be instantiated; the entire computation

can be completed through feedback reuse, achieving high resource utilization. The XOR operations in the compression phase alternate serially with the permutation, resulting in simple control logic.

## (2) Selection of State Size

The 1600-bit state is a core parameter of the algorithm; this size falls within the typical range of existing hash algorithms, balancing security strength and implementation efficiency. For software implementation efficiency, 1600 bits can be divided into 25 64-bit words or 50 32-bit words, adapting to register widths of mainstream processors. On 64-bit platforms, the state can be fully loaded into general-purpose registers, reducing memory access. For hardware implementation efficiency, 1600-bit registers have controllable area in FPGA/ASIC.

### 5.1.2 Software and Hardware Efficiency of Core Components

#### (1) Combinational Logic Characteristics of Permutation Function P

Permutation function P is designed as pure combinational logic with no internal state storage. This characteristic demonstrates efficiency advantages at both software and hardware levels.

- **Software level:** Pure combinational logic translates to a series of bitwise operations (XOR, shift, AND, OR, etc.) with no conditional branches, facilitating compiler optimization and easy parallelization using SIMD instructions.
- **Hardware level:** Combinational logic circuits have no clock dependency; complete transformation is accomplished in a single cycle; critical path length is determined only by gate-level delay, suitable for high-frequency operation.

#### (2) Efficiency of Bit-Sliced S-box Implementation

The 5-bit S-boxes adopted by this algorithm have logic that can be directly converted to bitwise operations, avoiding lookup table overhead. During bit-sliced implementation, the 1600-bit state can be decomposed into parallel processing of 25 64-bit states. This design achieves high parallelism in both software and hardware.

## 5.2 Performance Testing

### 5.2.1 Software Performance Testing

#### (1) Detailed Configuration of Implementation Platform

Litchi hash algorithm was tested and evaluated on a 64-bit platform using Intel Core i3-6100 CPU. Platform specifications are shown in Table 5-1.

Table 5-1 Software Evaluation Platform

| CPU                                                | Memory | Operating System | Programming Language | Compiler | Instruction Set | Cryptographic Library |
|----------------------------------------------------|--------|------------------|----------------------|----------|-----------------|-----------------------|
| Intel(R)<br>Core(TM)<br>i3-6100<br>CPU<br>@3.70GHz | 16GB   | Win7-64          | Standard C           | VS2022   | None            | None                  |

## (2) Implementation Methods

① Optimized implementation of nonlinear transformation: According to the scheme description, nonlinear transformation requires 21 slice operations.

$$B[0] = A[0]\&A[2]\&A[3] \oplus A[2]\&A[3]\&A[4] \oplus A[0]\&A[4] \oplus A[3]\&A[4] \oplus A[1];$$

$$B[1] = A[1]\&A[3]\&A[4] \oplus A[0]\&A[1] \oplus A[2];$$

$$B[2] = A[1]\&A[2] \oplus A[3];$$

$$B[3] = A[2]\&A[3] \oplus A[4];$$

$$B[4] = A[3]\&A[4] \oplus A[0]。$$

By reusing partial operations, the number of operations is reduced:

$$B[4] = A[3]\&A[4] \oplus A[0];$$

$$B[3] = A[2]\&A[3] \oplus A[4];$$

$$B[2] = A[1]\&A[2] \oplus A[3];$$

$$B[1] = B[4]\&A[1] \oplus A[2];$$

$$B[0] = B[4]\&B[3] \oplus A[1]。$$

It can be seen that only 10 slice operations are needed, reducing operations by more than half.

② Optimized implementation of linear transformation and transposition transformation: Linear transformation and transposition transformation are combined to reduce transposition operations.

$$T[i][j] = A[i][j] \oplus (E_i \ggg t_{i,j}) \oplus F_i$$

$$B[i][j] = T[(i + 2j) \bmod 5][(3i + 3j) \bmod 5]$$

Change to:

$$B[(4i + 4j) \bmod 5][(i + 3j) \bmod 5] = A[i][j] \oplus (E_i \ggg t_{i,j}) \oplus F_i$$

③ Addition of message update function to improve code flexibility.

④ Unification of absorption/release phases for the four function types (XOF, 512, 768, 1024) into a single function stage, reducing code redundancy while facilitating code management.

⑤ Other optimization methods: Macro-definition of low-level small module functions to reduce code call paths.

### (3) Performance Test Results

Table 5-2 Operation Speed of Litchi-512/768/1024 vs. SHA3-512

| Level | Data Length<br>(Bytes) | speed(Mbps) |            |             |          |
|-------|------------------------|-------------|------------|-------------|----------|
|       |                        | Litchi-512  | Litchi-768 | Litchi-1024 | SHA3-512 |
| S1    | 32                     | 416.5       | 416.1      | 403.5       | 370.6    |
| S2    | 128                    | 882.0       | 872.0      | 592.8       | 821.9    |
| S3    | 512                    | 1439.1      | 1204.4     | 810.4       | 839.8    |
| S4    | 1024                   | 1611.0      | 1313.7     | 862.8       | 905.7    |
| S5    | 4096                   | 1767.8      | 1329.5     | 908.7       | 957.7    |
| S6    | 8192                   | 1791.6      | 1362.6     | 910.9       | 959.9    |
| S7    | 16384                  | 1812.2      | 1369.5     | 913.2       | 965.0    |
| S8    | 65536                  | 1816.4      | 1374.2     | 921.8       | 968.1    |

### 5.2.2 Hardware Performance Testing

The Litchi hash algorithm outputs hash values of 512 bits, 768 bits, and 1024 bits, corresponding to three hardware implementation versions. All three versions were performance tested on both FPGA and ASIC.

#### (1) Detailed Configuration of Implementation Platform

Litchi hash algorithm was implemented and analyzed on xc7z100ffg900-2 FPGA chip and SMIC 28nm chip. Modelsim SE 10.1c was used for simulation. For FPGA testing, synthesis tool Vivado 2018.3 was used; for ASIC testing, Synopsys Design Compiler tool version syn\_v0\_2018.06-sp1 was used.

#### (2) Implementation Methods

Hardware implementation adopts a top-down approach; the algorithm top level contains only signal interconnections, with modules communicating via control and data signals. The hardware circuit logic block diagram is shown in Figure 5-1.

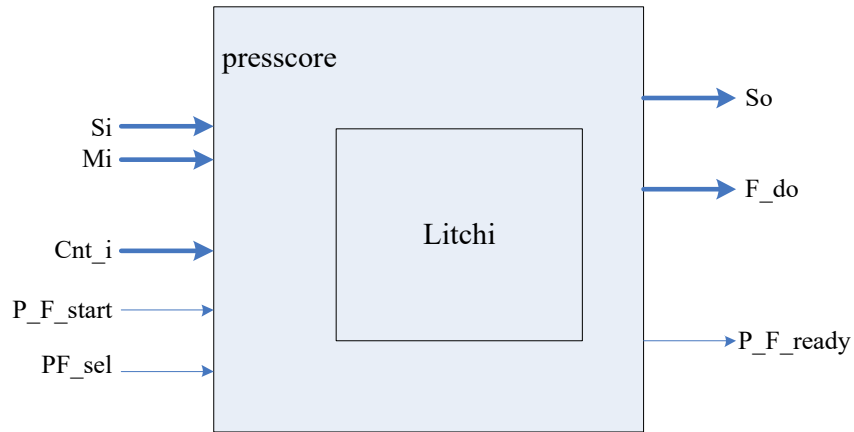


Figure 5-1 Hardware Circuit Logic Block Diagram

Logic top-level signal definitions are shown in Table 5-3:

Table 5-3 Logic Top-Level Signal Definitions

| Signal Name      | Signal Definition                                                        |
|------------------|--------------------------------------------------------------------------|
| <i>Si</i>        | Iterative state input                                                    |
| <i>Mi</i>        | Intermediate message block input                                         |
| <i>Cnt_i</i>     | Counter                                                                  |
| <i>P_F_start</i> | Processing start signal                                                  |
| <i>P_F_ready</i> | Processing completion signal                                             |
| <i>PF_sel</i>    | Function selection: 0 = message compression phase, 1 = hash output phase |
| <i>So</i>        | Iterative state output                                                   |
| <i>F_do</i>      | Hash result output                                                       |

### (3) Performance Test Results

FPGA test results are shown in Table 5-4:

Table 5-4 FPGA Test Results

| Version     | Resource usage |      |      | Performance simulation results |                         |                       |
|-------------|----------------|------|------|--------------------------------|-------------------------|-----------------------|
|             | Flip Flops     | LUTs | BRAM | Clock Period (ns)              | Short Packet 64B (Gbps) | Long Packet 1MB(Gbps) |
| Litchi-512  | 1670           | 4393 | 0    | 2.9                            | 10.27                   | 20.55                 |
| Litchi-768  | 1670           | 4352 | 0    | 2.9                            | 10.27                   | 15.41                 |
| Litchi-1024 | 1670           | 4404 | 0    | 2.9                            | 5.13                    | 10.27                 |

ASIC test results are shown in Table 5-5:

Table 5-5: ASIC Test Results

| Version     | Resource usage (GE) | Performance simulation results |                            |                          |
|-------------|---------------------|--------------------------------|----------------------------|--------------------------|
|             |                     | Clock Period (ns)              | Short Packet<br>64B (Gbps) | Long Packet<br>1MB(Gbps) |
| Litchi-512  | 41244               | 0.3                            | 99.34                      | 198.65                   |
| Litchi-768  | 41446               | 0.3                            | 99.34                      | 149.00                   |
| Litchi-1024 | 41697               | 0.3                            | 49.67                      | 99.33                    |

## 6 Features

### 6.1 Innovativeness

#### (1) Innovative design of a secure and efficient mode of operation

We have designed an improved mode of operation for the Litchi hash algorithm based on the Sponge- $F^{P,pd}$  construction proposed in [GHJ+25]. Compared to the widely used Sponge construction, it offers stronger (second) preimage resistance and higher message absorption efficiency. For example, for Litchi-512, its mode of operation achieves nearly twice the message absorption efficiency of SHA3-512.

#### (2) Innovative design of a simple and highly secure linear transformation

We have designed a parameterized 320-bit linear transformation, Theta, whose implementation depth is the same as that of Keccak's linear transformation, but whose branch number is increased from 4 to 12. A software implementation requires only 18 64-bit XOR operations and 10 cyclic shifts. By varying the parameters, different linear transformations can be constructed. Based on this linear transformation, we have designed a secure and simple 1600-bit internal permutation.

#### (3) Innovative design of a secure and simple internal permutation

The Litchi algorithm uses an internal permutation of 1600 bits, iterated over 16 rounds. All components of the round function are simple and efficient, and all operations can be processed in parallel. For example, the nonlinear transformation consists of 5-bit S-boxes; in a bitsliced implementation, the 1600-bit state is decomposed into 25 64-bit lanes for parallel processing. The transposition transformation incurs no implementation cost in either hardware or software. Moreover, the components of the internal permutation are closely interrelated, achieving high security. For example, full diffusion is achieved in 3 rounds, and the number of differentially active S-boxes over 3 rounds likely reaches around 60, which is a significant improvement over the round function of the Keccak permutation in terms of resistance to differential and linear attacks.

### 6.2 Simplicity

### (1) Simplicity of the mode of operation

Similar to the Sponge construction, the mode of operation is based on the internal permutation  $P$ , with other operations limited to simple XORs and state updates. There are no complex operations such as message expansion.

### (2) Simplicity of the internal permutation

All components of the internal permutation involve only bitwise AND, XOR, and cyclic shift operations, and each component is very simple, requiring few operations and having low depth.

## 6.3 Flexibility

### (1) Support for different hash output lengths

Litchi-512, Litchi-768, and Litchi-1024 share the same internal permutation  $P$ ; by adjusting parameters, different output lengths can be produced.

### (2) Ease of constructing XOF and MAC

Like Keccak, Litchi supports extendable output lengths, and a MAC can be readily constructed by concatenating a key with the message.

### (3) Excellent parallelism

The parallelism of the algorithm is mainly reflected in the following aspects:

- ① The nonlinear transformation supports parallel implementation.
- ② The linear transformation supports parallel implementation.
- ③ The extendable-output function (XOF) supports parallel output generation.

## 6.4 Implementation Performance Advantages on Different Platforms

### (1) Efficient software and hardware implementation

All operations of the Litchi algorithm are based on 64-bit words. Each component of the internal permutation  $P$  is simple and efficient. A single S-box requires only 10 basic operations; the linear transformation  $\theta_i$  requires only 18 64-bit XORs and 10 cyclic shifts; the transposition transformation incurs no additional implementation cost.

### (2) Excellent parallel implementation

The mode of operation supports parallel output when used as an extendable-output function (XOF), and the main cryptographic components of the internal permutation all support parallel processing.



## 7 Comparison with SHA-3 Algorithm

Both Litchi-512 and SHA3-512 support a hash output of 512 bits, and both use an internal permutation state of 1600 bits. Based on the same platform and with a comparable level of optimization, we implemented both algorithms in software and hardware. Table 7-1 provides a comparison of the two algorithms in terms of security and efficiency.

Table 7-1 Comparison of Litchi-512 and SHA3-512

| Algorithm  | Collision Resistance | Preimage Resistance | Second Preimage Resistance | Indifferentiability | Software Efficiency | Hardware Efficiency |
|------------|----------------------|---------------------|----------------------------|---------------------|---------------------|---------------------|
| Litchi-512 | $O(2^{256})$         | $O(2^{512})$        | $O(2^{512})$               | $O(2^{288})$        | 1816.4Mbps          | 20.55 Gbps          |
| SHA3-512   | $O(2^{256})$         | $O(2^{512})$        | $O(2^{512})$               | $O(2^{512})$        | 968.1Mbps           | 8.94 Gbps           |

*Note: The software platform is an Intel Core i3-6100 CPU, Windows OS. The hardware efficiency figures are from FPGA implementations using Vivado 2018.3 on an xc7z100ffg900-2 chip.*

## 8 References

- [ACMT25] Alagic, G., Carolan, J., Majenz, C., Tokat, S.: The Sponge Is Quantum Indifferentiable. pp. 2591-2630. IEEE Computer Society Press (2025).  
<https://doi.org/10.1109/FOCS63196.2025.00135>.
- [BC13] C.Boura and A.Canteaut, “On the influence of the algebraic degree of  $F^{-1}$  on the algebraic degree of GF, in IEEE Transactions on Information Theory, vol.59, no.1, pp.691-702, Jan.2013.
- [BDPV07] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. “Sponge functions”. Ecrypt Hash Workshop 2007. 2007. url: <http://sponge.noekeon.org/SpongeFunctions.pdf>.
- [BDPV08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. “On the Indifferentiability of the Sponge Construction”. In: EUROCRYPT2008. Vol. 4965. LNCS. Springer, 2008, pp. 181-197.
- [BDPV11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. “The

Keccak Reference”. Submission to NIST (Round 3).

- [DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schl  ffer.  
Ascon v1.2: Lightweight authenticated encryption and hashing. J. Cryptol.,  
34(3):33, 2021.
- [DM15] Dworkin, M.J., et al.: FIPS PUB 202-- SHA-3 standard: Permutation-based  
hash and extendable output functions (2015).
- [GHJ+25] Guo, C., Hu, K., Jiang, S., Fan, Y., Fu, Y., Preneel, B., and Wang, W.:  
Permutation-Based Hash from Non-Idealized Assumptions: Adding  
Feed-Forward to Sponge. Cryptology ePrint Archive (2025-1006).
- [HWL24] Lei H, Wang W, Zhang L. Symmetric twin column parity mixers and their  
applications[J]. IACR Transactions on Symmetric Cryptology, 2024, 2024(1).
- [Tod15] Yosuke Todo. “Structural Evaluation by Generalized Integral Property”. In:  
EUROCRYPT 2015. Vol. 9056. LNCS. Springer, 2015, pp. 287-314.